Implementation and Verification of a CAN/CAN FD Protocol Controller in VHDL

Mads Richardt (s224948)

May 18, 2026

Abstract

This thesis describes the design, implementation, and verification of a CAN/CAN FD protocol controller in VHDL, targeting high-reliability engine controller applications at Everllence. The controller complies with ISO 11898-1 and supports the CB, CE, FB, and FE frame formats with dual bit rate switching and Transmitter Delay Compensation (TDC) for the FD data phase. The design is structured around the ISO 11898-1 layered reference model, with the MAC, PCS, and FCE sub-layers implemented as independently testable modules and the LLC sub-layer specified but deferred. Of 38 derived requirements, 27 are verified against passing testbenches or code inspection. Of the remaining 11, eight fall outside the current scope pending `can_llc` integration, one is a lower-priority optional feature, and two represent identified future work: REQ-021 (error-injection under passive error conditions) and REQ-034 sub-claim 1 (lone-node bus re-integration). The design was synthesized on a Cyclone 10 LP FPGA target, using 4,608 logic elements (30% of device) with a worst-case fmax of approximately 127 MHz.

Approval

This thesis was prepared at the Department of Applied Mathematics and Computer Science (DTU Compute), Technical University of Denmark, in partial fulfilment of the requirements for the degree of Bachelor of Science in Engineering. The work was carried out in collaboration with Everllence, where the controller is intended for use in high-reliability engine controller applications.

It is assumed that the reader has a working knowledge of digital logic design and binary communication protocols. Familiarity with VHDL or a similar hardware description language is an advantage but is not strictly required to follow the architectural and verification discussions.

Mads Richardt (s224948)

Kgs. Lyngby, May 2026

Acknowledgements

Edward Alexandru Todirica, Associate Professor, DTU Compute. Thank you for supervision throughout the project and for valuable guidance and feedback.

Fredrik Kristensen, Everllence. Thank you for guidance throughout the project, code and report review, and feedback on the design and implementation.

Alex Fihl Hedegaard Nielsen, Everllence. Thank you for the sparring and advice, good company - and the many coffee machine trips that kept the work moving.

Contents

Abbreviations

Table 1: Abbreviations used in this report.

Abbreviation	Meaning
ACK	Acknowledgment
AD	ACK Delimiter
AEF	Active Error Flag
AI	Artificial Intelligence
BRS	Bit Rate Switch
CAN	Controller Area Network
CANH	CAN High bus wire
CANL	CAN Low bus wire
CB	Classic Base (frame format)
CBFF	Classic Base Frame Format
CC	CAN Classic
CD	CRC Delimiter
CE	Classic Extended (frame format)
CEFF	Classic Extended Frame Format
CRC	Cyclic Redundancy Check
DF	Data Frame
DLC	Data Length Code
DMA	Direct Memory Access
DUT	Device Under Test
ED	Error Delimiter
EOF	End of Frame
ESI	Error State Indicator
FB	FD Base (frame format)
FBFF	FD Base Frame Format
FCE	Fault Confinement Entity
FD	Flexible Data Rate
FDF	FD Frame bit
FE	FD Extended (frame format)
FEFF	FD Extended Frame Format
FPGA	Field-Programmable Gate Array
FSB	Fixed Stuff Bit
FSM	Finite State Machine
FTYP	Frame Type
HDL	Hardware Description Language
IDB	Identifier Base field (11-bit base ID)
IDE	Identifier Extension bit
IEXT	Identifier Extension field (18-bit extended ID)
IFS	Inter Frame Space
INT	Intermission
IP	Intellectual Property

Abbreviation	Meaning
ISO	International Organization for Standardization
LLC	Logical Link Control
LLM	Large Language Model
MAC	Medium Access Control
MIT	Massachusetts Institute of Technology (license)
MSB	Most Significant Bit
OD	Overload Delimiter
OF	Overload Flag
OSVVM	Open Source VHDL Verification Methodology
PCS	Physical Coding Sublayer
PE	Phase Error
PEF	Passive Error Flag
PS	Propagation Segment (PROP_SEG)
PS1	Phase Segment 1 (PHASE_SEG1)
PS2	Phase Segment 2 (PHASE_SEG2)
RF	Remote Frame
RRS	Reserved Remote Request Substitution bit (FD frames)
RTL	Register Transfer Level
RTR	Remote Transmission Request
RX	Receiver / Receive
SB	Stuff Bit
SBC	Stuff Bit Count
SJW	Synchronization Jump Width
SOF	Start of Frame
SP	Sample Point
SRR	Substitute Remote Request
SS	Sync Segment (SYNC_SEG)
SSP	Secondary Sample Point
ST	Suspend Transmission
TDC	Transmitter Delay Compensation
TEC/REC	Transmit Error Counter / Receive Error Counter
TQ	Time Quantum
TX	Transmitter / Transmit

List of Figures

Reading Guide

The report is structured in two parts. The first establishes context: the Introduction motivates the project and states the objectives; the Background (Section 2) and Protocol Overview (Section 4) provide technical foundations on CAN, CAN FD, and the VHDL/OSVVM toolchain. The second part is the technical contribution: Requirements, Verification Plan, Design and Architecture, Implementation, Verification and Results, and Synthesis form the core chapters, followed by Discussion and Conclusion.

Readers familiar with CAN and CAN FD may skip Section 2.1, Section 2.2, and Section 4. Readers familiar with VHDL and OSVVM may skip Section 2.3.

Source files, testbenches, verification plan, and tooling accompanying this document are listed in Section A.

1 Introduction

Industrial control systems for large marine engines demand communication protocols that combine fault tolerance, multi-master arbitration, and multi-decade service reliability. The Controller Area Network meets these demands, but as control system data requirements grow the bandwidth and payload limits of CAN Classic have become a bottleneck.

1.1 Motivation

Everllence (formerly MAN Energy Solutions) is a provider of propulsion and decarbonization solutions for the marine and energy industries, designing large two-stroke and four-stroke combustion engines for ship propulsion and power generation. This thesis was written within the engine controller division, where FPGA-based control hardware is developed and maintained for integration into Everllence's engine platforms.

The Controller Area Network (CAN), developed in the 1980s for automotive and industrial control, provides the fault-confinement properties that suit the reliability requirements of engine control applications [1]. Everllence's existing CAN infrastructure is built around a CAN Classic protocol controller deployed on an IO-extender board forming part of the Triton motor controller system. As control systems grow more data-intensive, the eight-byte payload and 1 Mbit/s ceiling of CAN Classic has become a practical constraint. CAN FD extends the maximum payload to 64 bytes and raises the data-phase bit rate beyond 8 Mbit/s while preserving the CAN Classic arbitration and fault-confinement architecture [2]. Thus, CAN FD is the natural migration path for Everllence's existing CAN infrastructure .

1.2 Existing CAN Controller

The starting point for this project is an existing CAN Classic controller developed internally at Everllence. The controller is implemented in VHDL and has been integrated into a production IO-extender FPGA design. It supports CAN Classic frames with both 11-bit (base) and 29-bit (extended) identifiers at bit rates up to 500 kbit/s, and has been verified through hardware bring-up on physical CAN buses. The initial version was developed by the author of the present document during an internship at Everllence and has since been extensively modified by other engineers at the company.

The top-level wrapper for the module instantiates a combined TX/RX frame FSM, a bit timing generator, a dynamic bit stuffer, a CRC-15 engine, and two Avalon-ST converters for frame serialization and deserialization.

The central component is the orchestrating FSM that handles both transmission and reception in a single process. It manages frame arbitration, bit-level TX and RX, stuff-bit error checking, CRC validation, ACK handling, and error flag generation. Fault confinement (error active, error passive, bus off node state transition) is handled in a separate process within the main FSM entity.

1.2.1 Limitations

While the existing controller is functional for CAN Classic, several areas of the existing design would require significant rework to support CAN FD.

Single bit rate domain. CAN FD requires switching between a nominal bit rate (used during the arbitration phase) and a faster data bit rate (used during the data phase), with Transmitter Delay Compensation (TDC) to account for the transceiver loop delay at the higher rate (Section 4.3). Adding dual bit rate support and TDC would require a fundamental redesign of the timing architecture.

Dynamic bit stuffing only. The bit stuffer implements the CAN Classic rule of inserting an inverse bit after five consecutive identical bits. CAN FD introduces a second stuffing mode - fixed bit stuffing - where a stuff bit is inserted at fixed intervals during the CRC field, and a Stuff Bit Count (SBC) field with Gray-coded parity is appended (Section 4.4). The existing stuffer has no mechanism for mode switching or SBC generation.

Single CRC engine. The controller has a single CRC engine. CAN FD requires three polynomials - CRC-15 for Classic frames, CRC-17 for FD frames with payloads up to 16 bytes, and CRC-21 for larger payloads (Section 4.5). A compliant receiver must run all three engines in parallel, since the correct polynomial depends on DLC and is not known until the control field is decoded. The data feed also differs between Classic and FD frames, requiring separate accumulation paths.

Coupled fault confinement. Error counting (TEC/REC) and node state transitions are implemented in the same source file as the frame FSM, with no clean sub-layer boundary between them (Section 4.6). ISO 11898-1 defines fault confinement as a distinct cross-cutting entity, and separating it as an independently testable module both conforms closer to the standard's reference model (Section 4.1) and simplifies verification.

Format-dependent frame structure. CAN Classic and CAN FD frames diverge in the control and data phases, where FD introduces format-specific fields with no Classic equivalent (Section 4.2). The existing FSM has no clean mechanism to handle this divergence across four frame variants.

1.2.2 Decision to Redesign

The rework required across bit timing, bit stuffing, CRC, and frame format complexity is large enough to justify a clean-slate redesign structured around the ISO 11898-1 layered architecture (LLC, MAC, PCS, FCE, Section 4.1) with independently testable subcomponents, rather than retrofitting FD support onto a design not originally built with those boundaries.

1.3 Existing CAN FD IP Cores

Before committing to an in-house redesign, the available CAN FD controller IP cores were evaluated. Table 2 summarizes the candidates, spanning both open-source and commercial offerings.

1.3.1 Open-Source Implementations

CTU CAN FD [3] is the only mature open-source CAN FD controller available as synthesizable HDL. Developed at the Czech Technical University in Prague, it is written in VHDL, licensed under MIT, and has been conformance-tested against ISO 16845-1 [4]. The controller includes a full TX and RX pipeline with up to four TX buffers, acceptance filtering, timestamping, and a register interface with DMA support.

1.3.2 Commercial Implementations

Bosch M_CAN [5] is the reference CAN FD controller developed by Bosh. M_CAN is the IP core embedded in most automotive micro controllers. It is licensed under a non-disclosure agreement with per-design royalty fees.

AMD/Xilinx CAN FD [6] is a soft IP core included in the Vivado Design Suite. It provides an AXI4-Lite register interface with up to 32 acceptance filters, TX mailboxes, and RX FIFOs. It is device-locked to AMD/Xilinx FPGAs and cannot be ported to other targets.

Technology-independent RTL cores. CAST CAN FD [7] is a technology-independent CAN FD IP core licensed per-design with an upfront fee. Major EDA vendors including Synopsys (DesignWare) and Cadence offer similar ASIC-targeted cores under commercial licensing programs.

Table 2: Survey of available CAN FD controller IP cores.

Implementation	Language	License	Scope	Conformance Tested
CTU CAN FD [3]	VHDL	MIT	Full node (TX+RX, buffers, DMA)	ISO 16845-1
Bosch M_CAN [5]	HDL (non-disclosure agreement)	Per-design royalty	Full node	Yes (reference)
AMD CAN FD [6]	HDL	Vivado-included	Full node	Yes
CAST CAN FD [7]	HDL	Per-design fee	Full node	Yes

1.3.3 Rationale for In-House Development

None of these solutions satisfies Everllence's combined requirements for safety-critical marine engine control. The disqualifying factors span IP ownership, verification authority, architectural scope, integration with existing infrastructure, and platform independence - each addressed in turn below.

IP ownership and supply chain independence. Everllence's engine controllers carry service commitments of up to thirty years. Commercial IP cores introduce a licensing dependency on an external vendor over that full horizon - vendors may discontinue support, change licensing terms, or be acquired. Owning the RTL outright eliminates this exposure and ensures that the design can be maintained, ported, and modified without third-party approval for the full product lifetime. The open-source CTU CAN FD avoids the licensing risk, but using it still means adopting a codebase whose architecture, naming conventions, and design decisions were made for a different context.

Verification authority. In safety-critical domains, the verification evidence must be traceable from standard requirements to RTL assertions and testbench results. Adopting a third-party core means inheriting its verification artifacts rather than producing them. Everllence's verification methodology requires full control over the verification plan, the testbench architecture, and the assertion coverage.

Architectural scope. All available IP cores implement a complete CAN node - TX and RX pipelines, message memory, acceptance filtering, buffer management, register interfaces, and in some cases DMA controllers. Everllence's application requires only the protocol engine - the module that converts between byte-level frame data and the serial bus - which is exactly the scope of the existing CAN Classic controller. The higher-level buffering and filtering logic already exists in Everllence's FPGA infrastructure. Adopting a full-node IP core would introduce unnecessary complexity and area overhead, and stripping the unused subsystems to fit the existing architecture offsets the benefit of using a pre-built core.

Integration with existing infrastructure. Everllence's FPGA designs use a specific Avalon-ST streaming interface for inter-module communication and established conventions for signal naming and module boundaries. A third-party core would require an adaptation layer to bridge its native interface to the existing infrastructure. The in-house design uses Everllence's interface conventions natively, eliminating this integration overhead.

Platform independence. The AMD/Xilinx CAN FD core is locked to Xilinx devices. The Bosch M_CAN and other commercial cores are delivered as technology-specific netlists or encrypted HDL for a particular target. The in-house design is written in portable VHDL-93, synthesizable on any FPGA platform ensuring that the IP remains usable if Everllence changes FPGA vendors.

1.4 Problem Statement

The architectural limitations of the existing controller (Section 1.2.1) and the unsuitability of available third-party IP cores (Section 1.3.3) together motivate a clean-slate CAN FD protocol controller conforming to ISO 11898-1. No existing solution combines full IP ownership, a targeted data-link-layer scope matching Everllence's integration requirements, and native compatibility with Everllence's Avalon-ST interface conventions and VHDL Code Standard. The design described in this report addresses that gap directly.

1.5 Objectives

1. Implement a CAN/CAN FD protocol controller in VHDL, compliant with ISO 11898-1 [8] and supporting the CB, CE, FB, and FE frame formats.
2. Establish a structured requirements framework with traceability from ISO 11898-1 to testbench evidence, covering all implemented modules.
3. Produce an RTL design integrated via Avalon-ST interfaces into Everllence's existing FPGA infrastructure.

2 Background

This section covers the two technical foundations that the rest of the report builds on. The first is the CAN and CAN FD protocol at the level of motivation and architecture: its bus model, fault-confinement properties, and the bandwidth extensions introduced by CAN FD. The protocol mechanisms referenced by individual requirements - bit timing, stuffing, CRC, and error handling - are covered in depth in Section 4. The second foundation is the VHDL-93 and OSVVM toolchain used for RTL implementation and simulation. Readers already familiar with both may proceed directly to Section 3.

2.1 CAN Classic

The Controller Area Network (CAN) is a serial communication bus developed by Bosch in 1986 [1] to connect electronic control units in automotive environments without a central host computer. Where point-to-

point wiring and star-switched architectures require a dedicated conductor between every communicating pair, CAN uses a shared two-wire differential bus on which all nodes broadcast simultaneously and arbitrate access without any designated bus master (see Figure 1). Any node may initiate a transmission at any time. Contention is resolved by a non-destructive bitwise arbitration in which the transmitter with the lower-priority identifier detects the collision and silently withdraws, leaving the winner's frame intact. Differential signaling on a twisted pair (ISO 11898-2 physical layer) provides strong common-mode noise rejection - a practical necessity in the electrically harsh environment of an engine bay or industrial cabinet.

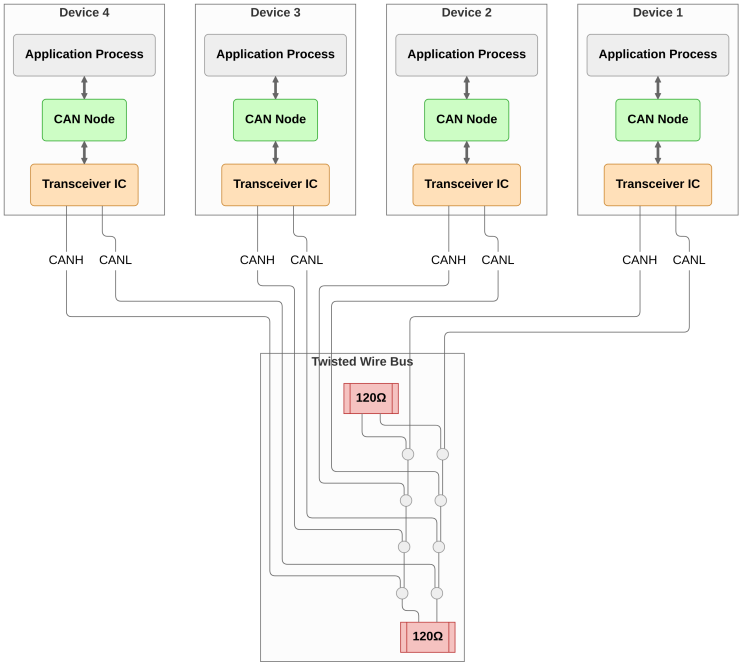

Figure 1: CAN bus with four nodes on a shared differential two-wire bus.

CAN's error-handling architecture is a distinguishing feature relative to simpler serial protocols. Five complementary error detection mechanisms operate concurrently on every transmitted frame: bit monitoring, frame format checking, cyclic redundancy checking, acknowledgment checking, and bit stuffing violation detection. Charzinski showed that under a two-state channel model the residual error probability for an eight-byte frame in a ten-node network is bounded by approximately $3.5 \times 10^{-9} \cdot q_{\text{bad}}$ per frame, where q_{bad} is the probability of a frame being transmitted during a bad channel period [9]. This figure is several orders of magnitude lower than contemporary automotive bus alternatives such as VAN and SCP evaluated under the same model [9]. A fault confinement mechanism tracks each node's error history and automatically escalates from error active through error passive to bus off, electrically isolating a persistently faulty node from the bus without disrupting communication between healthy nodes. Together these properties made CAN the protocol of choice for safety-relevant in-vehicle networks. Adoption subsequently spread to industrial automation, medical devices, and aerospace ground support equipment.

The fault-confinement and multi-master properties that distinguished CAN from its contemporaries were deliberately preserved in CAN FD: Hartwich's design goal was to extend payload and bit rate while leaving the arbitration and fault-confinement architecture unchanged, specifically to retain the properties that made CAN suitable for safety-relevant distributed control networks [2].

2.2 CAN FD

CAN's original data payload was capped at eight bytes per frame, limiting raw throughput to around 1 Mbit/s. As embedded control applications became more data-intensive, this ceiling became a practical constraint. CAN FD (Flexible Data Rate), introduced by Bosch in 2012 [2] and incorporated into ISO 11898-1 in 2015, extends the maximum payload to 64 bytes and introduces a separate higher-speed data phase with bit rates of 8 Mbit/s or beyond, while preserving the CAN Classic arbitration phase and the fault-confinement architecture unchanged. The bandwidth constraint imposed by arbitration - where signal propagation time between all nodes limits the bit rate - applies only during the arbitration phase when multiple nodes may simultaneously drive the bus. Once arbitration is resolved and a single transmitter controls the bus, the bit rate can be increased freely, limited only by transceiver slew rate and oscillator stability [2]. Hartwich demonstrated average data rates of 2.5 Mbit/s achievable with standard CAN transceivers, matching the effective payload of a low-speed FlexRay network [2]. At high data-phase bit rates the transceiver's TX-to-RX loop delay (approximately 100 ns for a typical CAN FD transceiver such as the TCAN1042 [10]) may exceed one bit time, requiring Transmitter Delay Compensation (TDC) to correctly position the secondary sample point used for bit-error monitoring. Mutter showed that for data-phase to arbitration-phase bit rate ratios below approximately 9, CAN FD accepts the same oscillator tolerance as CAN Classic, preserving compatibility with commodity crystal oscillators [11].

CAN FD also strengthens the error detection architecture. The longer payloads require stronger CRC polynomials: a 17-bit BCH polynomial covers frames up to 16 data bytes and a 21-bit polynomial covers frames up to 64 bytes, both maintaining Hamming distance 6 [2]. A known weakness in CAN Classic, where two bit errors that generate and eliminate stuff conditions can pass undetected through the CRC [9], is addressed in CAN FD by including dynamic stuff bits in the CRC data feed and introducing the Stuff Bit Count (SBC) field. These improvements together reduce the residual error probability in the worst-case error class by several orders of magnitude compared to CAN Classic [12]. The governing standard for this project is ISO 11898-1 [8], which specifies both CAN Classic and CAN FD data link layer and physical signaling requirements.

2.3 Tools and Language

The RTL source is implemented in VHDL-93. Everllence's synthesis toolchain uses Quartus Prime, which does not fully support VHDL-2008 constructs in synthesis, making VHDL-93 the practical upper bound for synthesizable RTL. Testbenches are written in VHDL-2008 to support the OSVVM verification framework [13], which requires VHDL-2008 language features. SystemVerilog with UVM is the dominant industry alternative for RTL implementation and verification at this scale. The choice here follows company convention rather than a project-level technical comparison. Riviera-PRO is used for simulation. Sigasi is used for linting and language-aware editing. Waveform figures are captured in GTKWave, timing diagrams are drawn in WaveDrom, and architecture diagrams in Mermaid.

3 Requirements

The Everllence coding constraints establish the implementation framework - port type restrictions, naming conventions, and testbench structure - that applies uniformly to all in-house FPGA modules. The 38 protocol requirements are derived from ISO 11898-1 normative obligations through an AI-augmented extraction pipeline and distilled manually into independently verifiable entries, each carrying a source clause reference, a priority rating, and a verification method. Together they bound the design space before any architectural decisions are made.

3.1 VHDL Code Standard and Design Constraints

The constraints on this project come from two distinct sources. Two are specific to this project's integration context: the Avalon-ST host interface and the mandatory use of the IP library CRC block. The remainder are drawn from Everllence's VHDL Code Standard and apply uniformly to all in-house FPGA modules.

3.1.1 Project-specific Infrastructure Requirements

1. **Avalon-ST user interface:** The CAN controller's external interface to the host system must use the Avalon-ST streaming protocol [14] (data, valid, ready, sop, eop). The requirement applies specifically to the boundary between the CAN controller and its user.
2. **IP library CRC block:** Everllence maintains a reusable, parameterised CRC generator (`gen_crc`) in its IP library. This module must be used for all CRC computations.

3.1.2 File Structure and Naming

1. **Per-module file structure:** Each module must be organized into a fixed directory layout: `hdl_src/` for RTL source files, `hdl_tb/` for testbench files, and `test_case/` for waveform configuration. One entity per VHDL file, with the filename matching the entity name. Every entity requires a dedicated testbench, unless it is instantiated exclusively as a submodule of a fully tested parent.
2. **Entity port types:** Entity ports are restricted to `std_logic`, `std_logic_vector`, and records or arrays of these types. Modes are restricted to `in` and `out`.
3. **Naming conventions:** A mandatory prefix/suffix scheme applies to all VHDL identifiers: types (`t_`), constants (`c_`), generics (`gc_`), processes (`p_`), functions (`f_`), packages (`pk_`), state variables (`s_`), entity inputs (`_i`), and entity outputs (`_o`).

3.1.3 RTL Coding Style and Verification

1. **RTL design rules:** Synchronous processes must be sensitive to the clock only. Reset must be synchronous and initialize all control registers. FSMs are preferably implemented as single-process designs where all signal assignments are derived from the current state, with an explicit other-state that returns to a known safe state.
2. **Testbench requirements:** Testbenches must follow a black-box testing model and test cases must be ordered as: reset tests first, then a normal-usage test, then all remaining tests.

3.2 From Specification to Structured Requirements

The requirements engineering process addressed two key objectives [15]:

1. Extracting a clear and actionable set of requirements that could serve as a starting point for the design phase.
2. Establishing a clear, traceable link between the ISO 11898-1 specification and the verification environment.

Both objectives are complicated by the source material: normative requirements are distributed across subsections, often restated from different perspectives, and interspersed with explanatory text. The standard compounds this by bundling multiple obligations into single clauses, interspersing normative `shall` statements with informative rationale prose, and repeating equivalent obligations from both transmitter and receiver perspectives.

The AI-augmented pipeline shown in Figure 2 was designed to address these extraction challenges systematically. The first step was converting the ISO 11898-1 PDF to Markdown - a format that can be efficiently searched and ingested by LLMs. The resulting Markdown file was then fed to a Claude Sonnet 4.6 LLM agent, which was prompted to extract all normative statements - sentences containing words like "shall", "should", "must", and their corresponding negations.

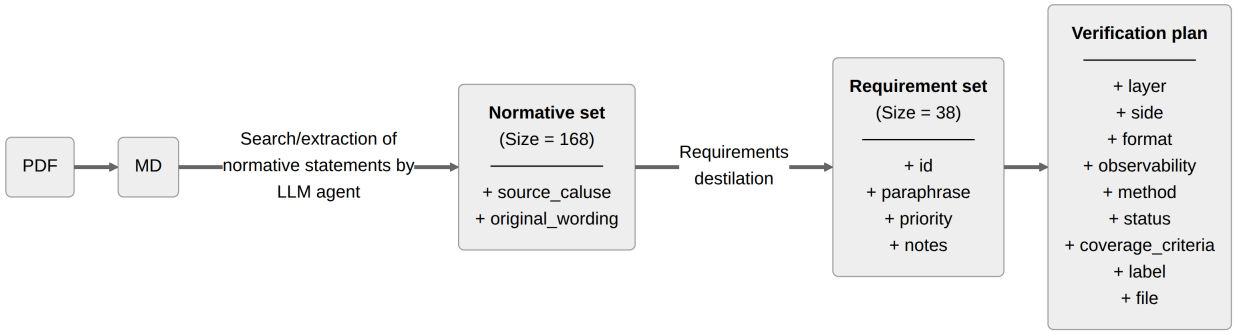

Figure 2: Pipeline generating the `verification_plan.toml` artifact from the ISO 11898-1 standard.

This process yielded a raw set of 168 normative statements linked to the ISO standard sections from which they were extracted. The normative set was then manually reviewed, consolidated, and distilled into a final set of 38 requirements (reproduced in Section C).

The requirements set is stored as a configuration file (`verification_plan.toml`), one `[[requirement]]` block per entry. To support ongoing AI-assisted refinement without the risk of silent data corruption, a custom Model Context Protocol server (`verification_plan_manager.py`) was developed alongside the plan. The server exposes query, update, insert, and statistics operations as tool calls that the AI coding agent can invoke directly within its development environment. Each write operation targets a single requirement entry and validates field values against the schema before committing. This makes it structurally impossible for the agent to silently drop entries, fabricate field values, or corrupt the file syntax - failure modes that arise inevitably when an LLM is asked to rewrite a large structured file in one operation. During the requirements phase, the agent worked exclusively with the five fields relevant at this stage:

- **`source_clause`**: The ISO 11898-1 section reference (e.g. §6.6.13.1). This is the traceability anchor - every requirement links back to the clause from which it was distilled, making it possible to verify the requirements set against the standard during review.
- **`original_wording`**: Verbatim ISO text for the relevant clauses. Preserving the source wording prevents paraphrase drift and provides a fallback for resolving ambiguity during implementation.
- **`paraphrase`**: A concise, implementer-facing restatement of the requirement. Where the ISO prose bundles multiple obligations into a single clause, the paraphrase enumerates them as numbered sub-claims, each independently verifiable.
- **`priority`**: A three-level rating - P1 (need-to-have, derived from "shall" obligations and core correctness), P2 (verified in the normal cycle), or P3 (optional, derived from "should" clauses or implementation-dependent features). Priority drives implementation sequencing and scope decisions when schedule is constrained. Of the 38 requirements, 32 are rated P1, four are P2, and two are P3. Every requirement not rated P1 was explicitly demoted. The rationale for each demotion is given in Table 3.
- **`notes`**: Residual clarifications not resolved by the paraphrase - implementation constraints, out-of-

scope markers, or known ambiguities flagged for design review.

Table 3: Priority demotion rationale for all requirements not rated P1.

ID	Topic	Priority	Demotion rationale
REQ-002	LLC TX request and abort timing	P2	The 2-SOF processing window is a responsiveness guarantee, not a correctness constraint. A node that transmits eventually but outside this window sends valid frames.
REQ-011	Remote frame	P2	A data-only node is a valid CAN implementation. Remote frame support is a distinct feature subset not required for basic interoperability.
REQ-015	ESI bit transmission	P2	ESI communicates the node's error state as an informational signal. Incorrect ESI does not abort a frame or trigger a protocol error at any receiver.
REQ-036	Error signaling enable	P2	Error signaling itself is covered by P1 requirements. This requirement concerns only the existence of a configurable disable mode, which is an optional operational feature.
REQ-004	Frame acceptance filtering	P3	ISO uses advisory "should" language. Acceptance filtering is also an application-layer concern above the LLC boundary verified here.
REQ-037	DLC padding	P3	Padding with 0xCC applies only when the implementation exposes a configurable maximum-data-byte restriction. The feature may be waived entirely if that restriction is not implemented.

3.3 AI-Assisted Extraction: Utility and Limitations

The AI-assisted workflow delivered value in two distinct phases of the project, with very different characteristics in each.

In the extraction phase, the LLM agent earned its keep by bootstrapping and linking the initial normative statement set. Having a fully populated and linked starting point - even one requiring substantial revision - gave the manual review process a concrete artifact to work from. The time saving from the extraction itself was, however, marginal. The agent's output had to be reviewed statement by statement, which is functionally similar to extracting requirements manually in the first place. The primary benefit of the AI-assisted approach in this phase was therefore not efficiency, but rather the increased consistency of an automated pass over the full standard text.

The distillation step that followed - consolidating 168 raw normative statements into 38 prioritized, independently verifiable requirements - required substantial manual effort that the AI could not replace. Deciding which statements address the same underlying obligation, how to bound each requirement so that it is independently verifiable, and how to assign priority in a way that is defensible against the standard text are judgment calls that depend on understanding the protocol at an implementation level.

REQ-026 (PCS synchronization, §7.3.5.1–7.3.5.4) illustrates the challenge well.

Extracted normative statements (§7.3.5.1–7.3.5.4):

1. *"Only one synchronization within one bit time (between two sample points) shall be allowed. After an edge is detected, synchronizations shall be disabled until the next time the bus state detected at the sample point is recessive."*

2. *"An edge shall cause synchronization only if the bus state detected at the previous sample point was recessive. If a transmitter uses transmitter delay compensation, also the first detected edge from recessive to dominant after the sample point of the CRC delimiter shall cause synchronization. An edge with a positive phase error shall not cause synchronization in a node sending a dominant bit."*
3. *"Hard synchronization shall be performed: on edges during inter-frame space (with the exception of the first bit of intermission); when a node is in bus-integration state; at the edge between the FDF bit and the following dominant res bit inside an FD frame; when a node is a receiver of an XL frame, at the edge between the XLF bit and the following dominant resXL bit; at the edge between the DH1 and DH2 bits and the DL1 bit; at the edge between the DAH or AH1 bit and the AL1 bit."*
4. *"All other recessive-to-dominant edges fulfilling rules a) and b) shall be used for resynchronization with one exception: a node transmitting an FD frame or an XL frame shall not synchronize while it transmits the data phase of that frame."*
5. *"After a hard synchronization, the bit time shall be restarted with Sync_Seg completed. Hard synchronization shall not be limited by the synchronization jump width."*
6. *"When the magnitude of the phase error is less than or equal to the synchronization jump width, the effect of a resynchronization shall be the same as a hard synchronization."*
7. *"When the magnitude of the phase error is larger than the synchronization jump width: if positive, Phase_Seg1 shall be lengthened by the synchronization jump width; if negative, Phase_Seg2 shall be shortened by the synchronization jump width."*

Statements 3 and 4 each embed CAN XL conditions within otherwise in-scope obligations. Statement 3 lists XL-specific hard-sync trigger points - XLF, DH1/DH2/DL1, and DAH/AH1 frame boundaries - alongside the FDF trigger that applies here. Statement 4 suppresses resync during the data phase of both FD and XL frames, but only the FD clause is in scope. These sub-clauses must be identified and excluded without dropping the surrounding obligations that apply to CB, CE, FB, and FE frames. Statement 4 also contains a dangling cross-reference: "fulfilling rules a) and b)" refers to labeled items defined within §7.3.5.2, but the extraction captured only the prose sentences, discarding the letter identifiers. The extracted set contains no entry labeled "rule a)" or "rule b)", so the reference cannot be resolved without returning to the original ISO clause - an omission that the AI extraction did not flag. The remaining five statements interleave hard sync and resync rules across four sub-clauses, requiring the two mechanisms to be separated and their interaction - phase error not exceeding SJW collapses resync to the same effect as hard sync - made explicit. The resulting `paraphrase` field is presented below, with the remaining requirement plan fields in Table 4.

Paraphrase:

1. One synchronization per bit time, re-enabled after the next recessive sample point.
2. An edge qualifies only if the previous sample point was recessive and no positive phase error exists while sending dominant. The first recessive-to-dominant edge after the CRC delimiter also qualifies when TDC is active.
3. Hard synchronization: IFS edges (except the first intermission bit), bus-integration state, and the FDF-to-res transition. Bit time restarts with Sync_Seg completed, unconstrained by SJW.
4. All other qualifying edges cause resynchronization, except for the FD data-phase transmitter. Positive error: Phase_Seg1 += SJW. Negative error: Phase_Seg2 -= SJW. Error not exceeding SJW: same effect as hard synchronization.

Table 4: REQ-026 distilled from seven extracted normative statements.

Field	Value
ID	REQ-026
Source	§7.3.5.1, §7.3.5.2, §7.3.5.3, §7.3.5.4
Priority	P1
Notes	The implementation is stricter than ISO: `mac_i.transmitting` suppresses all synchronization unconditionally, not just resync on positive phase error. This is safe since a transmitter is the timing reference for all receivers.

The MCP server interface proved genuinely useful throughout the design, implementation, and verification phases that followed extraction. As implementation decisions were made, requirements were refined - paraphrases sharpened, notes extended, observability classifications updated, and traceability fields populated. The AI coding agent performed these updates directly through the MCP tool calls, targeting individual requirement fields against a schema-validated store. The alternative - asking an agent to rewrite the full TOML file each time a requirement changed - would have introduced silent data corruption risks at every edit. The narrow, validated write interface made incremental AI-assisted maintenance of the verification plan safe and practical across all three project phases.

The 38 requirements, each linked to its ISO source clause and assigned a priority, define the scope of what must be implemented and verified. They also function as a structured map to the protocol: every requirement points to a mechanism that must be understood before implementation can begin. Section 4 provides that understanding - covering the sub-layer model, frame formats, bit timing, bit stuffing, CRC, and error handling.

Generative AI was also used during the preparation of this report. Claude Code (Anthropic) was used as a writing aid throughout: reviewing phrasing and tightening prose. All edits proposed by the tool were only applied after explicit author approval. Technical content, judgments, and decisions are the author's own. Claude Code was additionally used as a coding aid during implementation and verification: reviewing VHDL for correctness and assisting with debugging. All design decisions, architectural choices, and VHDL source files are the author's own work.

4 CAN and CAN FD Protocol Overview

Each requirement in Section 3 refers to a specific protocol mechanism. The sub-layer model, frame formats, bit timing, stuffing, CRC, and error handling are covered here, each cross-referenced to the relevant REQ-NNN entries. Readers familiar with ISO 11898-1 may skip to Section 5.

4.1 Layered Reference Model

ISO 11898-1 structures the CAN data link layer into three functional sub-layers and a cross-cutting Fault Confinement Entity (FCE) (see Figure 3):

- **LLC (Logical Link Control)**: accepts frame requests from the host application, applies retransmission policy on error or lost arbitration, and supplies frames to the MAC in serialized form.
- **MAC (Medium Access Control)**: encodes and decodes the frame bit-by-bit - performing bit stuffing and destuffing, CRC generation and checking, and acknowledgment handling - and governs bus access arbitration.

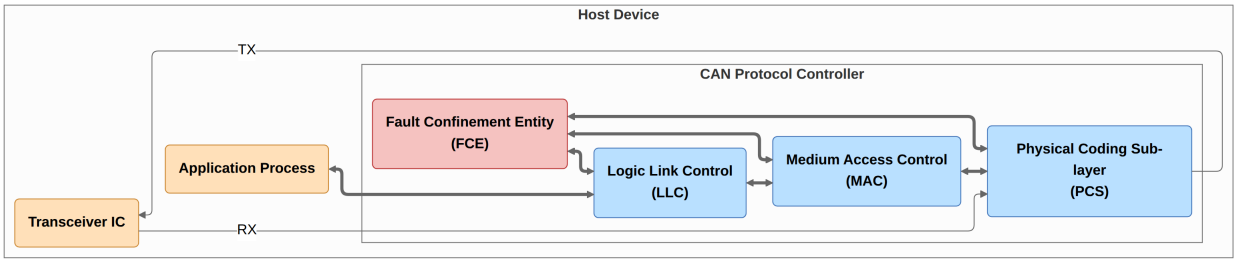

Figure 3: ISO 11898-1 CAN node reference model showing the LLC, MAC, PCS sub-layers and cross-cutting FCE.

- **PCS (Physical Coding Sublayer)**: manages bit timing, clock synchronization (including Transmitter Delay Compensation for FD data phase), and the sample/drive interface to the physical transceiver.
- **FCE (Fault Confinement Entity)**: maintains Transmit Error Counter (TEC) and Receive Error Counter (REC), escalating the node's error state from error active through error passive to bus off as error counts accumulate.

In the implementation described in this report, each sub-layer maps to a dedicated VHDL module, and the sub-layer interfaces become the port records connecting those modules (Section 6). Table 5 shows the requirement-to-layer assignment for all 38 requirements.

Table 5: Requirement-to-layer assignment.

Sub-layer	Requirements
LLC	REQ-001–005, REQ-032, REQ-037
MAC	REQ-006, REQ-008, REQ-010–013, REQ-015–019, REQ-021–023, REQ-031, REQ-033, REQ-035–036, REQ-038
PCS	REQ-024–027
FCE	REQ-028–030
System	REQ-007, REQ-009, REQ-014, REQ-020, REQ-034

4.2 Frame Types and Formats

CAN defines two classes of frames: CAN Classic (CC) and CAN FD (FD). Within each class, frames may carry either an 11-bit base identifier or a 29-bit extended identifier, giving four frame formats: CB (Classic Base), CE (Classic Extended), FB (FD Base), and FE (FD Extended), as shown in Figure 4 (REQ-038). Classic frames (CB and CE) additionally support remote frame variants (RTR=1, no data field), giving six bus frame types in total. CAN XL frames are out of scope for this project.

A CAN Classic frame consists of Start of Frame (SOF), Arbitration field (identifier, RTR, IDE), Control field (DLC), Data field, CRC field, ACK slot and delimiter, End of Frame (EOF), and Intermission. SOF is a single dominant bit that marks the beginning of a frame and triggers hard synchronization in all receiving nodes (REQ-038). Within the arbitration field, bits are transmitted MSB first (REQ-033). The RTR bit distinguishes data frames from remote frames, being dominant for data frames and recessive for remote frames (REQ-038). In extended frames, an SRR placeholder bit transmitted recessive precedes the IDE bit (REQ-038). The DLC encodes the number of data bytes using the four-bit mapping defined in ISO 11898-1 (REQ-032, REQ-037). After the data and CRC fields, the ACK slot carries a dominant bit driven by every receiver that has successfully validated the frame CRC - the transmitter monitors this slot and reports an acknowledgment error if no dominant bit is received (REQ-014). The frame is delimited by seven recessive EOF bits followed by three recessive intermission bits (REQ-038, REQ-008).

A CAN FD frame shares the same structure through the arbitration phase and then introduces FD-specific control fields. The FDF bit distinguishes an FD frame from a Classic frame. A recessive FDF triggers the FD control field sequence including reserved bits, BRS, and ESI (REQ-038). The BRS (Bit Rate Switch) bit

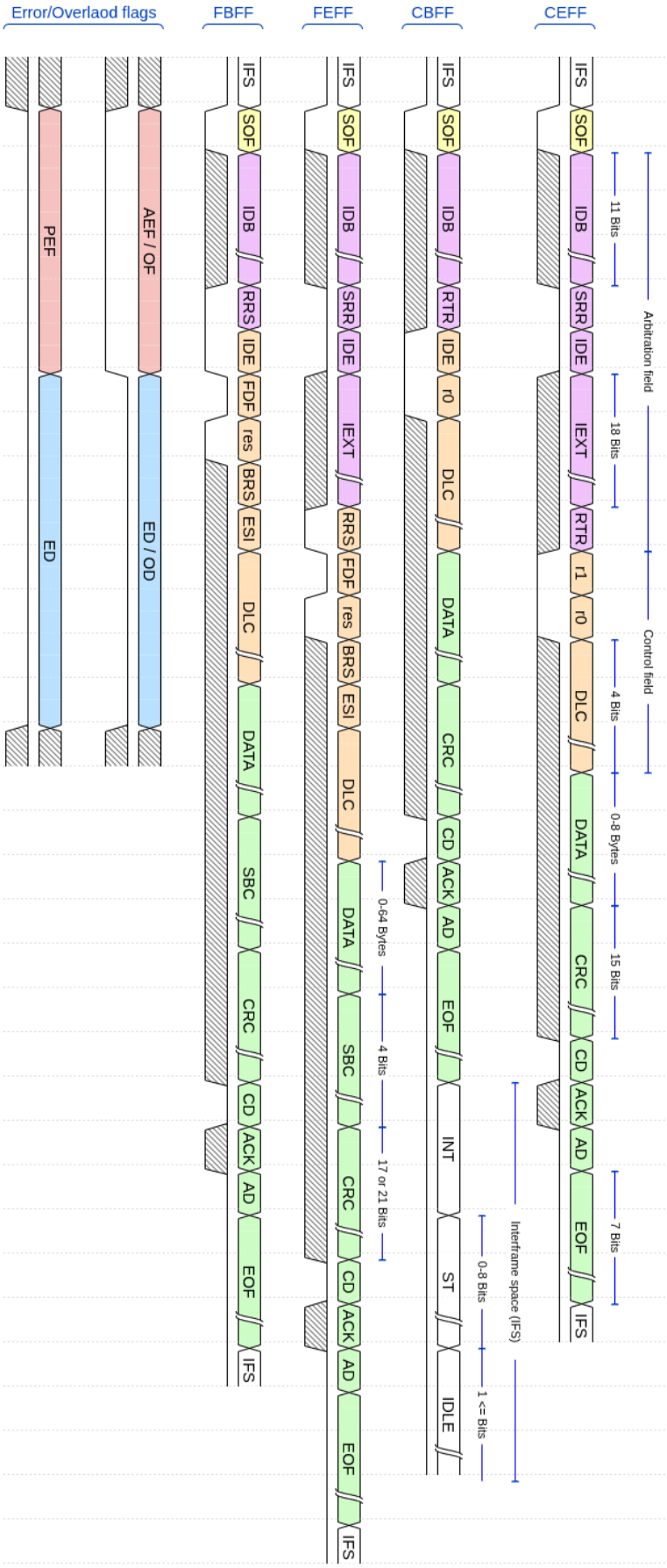

Figure 4: Frame formats for the four in-scope frame types (CB, CE, FB, FE) and the error and overload flags. Field widths are annotated per ISO 11898-1.

4.3 Bit Timing and Flexible Data Rate

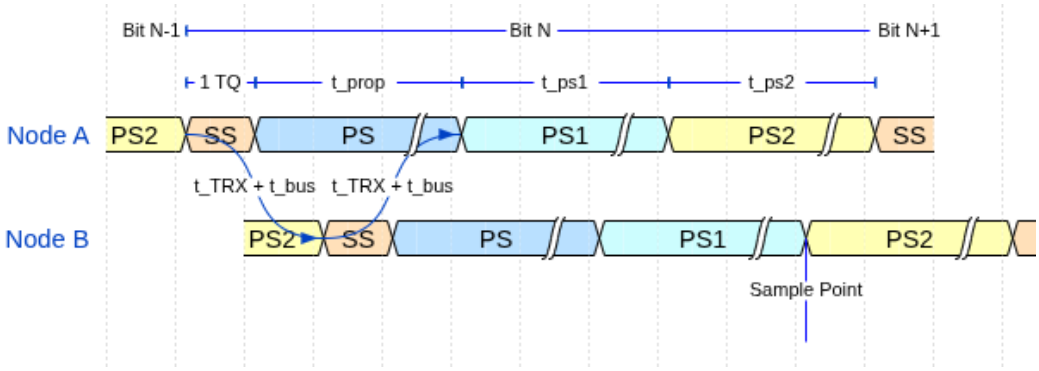

Figure 5: CAN bit time structure showing the four segments (SS, PS, PS1, PS2), the sample point, and propagation delays on a two-node bus.

Every CAN bit period is divided into four non-overlapping time segments measured in Time Quanta (TQ), where one TQ equals the period of the prescaled system clock (see Figure 5). CAN FD extends CAN Classic with an independently configured data-phase bit rate. A CAN FD node therefore maintains two independent sets of segment parameters, one for the nominal rate and one for the data rate (REQ-024):

- **Sync Segment (SYNC_SEG)**: one TQ. The point at which the bus is expected to produce a recessive-to-dominant edge after synchronization.
- **Propagation Segment (PROP_SEG)**: compensates for round-trip signal propagation delay on the bus and in the transceiver. It shall be programmed to be at least as long as twice the maximum bus propagation delay.
- **Phase Segment 1 (PHASE_SEG1)**: immediately precedes the sample point. Can be lengthened by the resynchronization mechanism to absorb positive phase errors.
- **Phase Segment 2 (PHASE_SEG2)**: follows the sample point to the end of the bit. Can be shortened to absorb negative phase errors.

The **sample point** falls at the PHASE_SEG1 / PHASE_SEG2 boundary. Every receiver samples the bus exactly once per bit at this point. The sample point position - expressed as a percentage of the total bit time - is a configuration parameter traded off against bus length, node count, and oscillator tolerance.

Resynchronization corrects for accumulated phase error between a receiver's local oscillator and the transmitter's bus edges. Hard synchronization forces a full re-alignment on the SOF falling edge at the start of each frame. During the frame, resynchronization adjusts PHASE_SEG1 or PHASE_SEG2 by up to the configured Synchronization Jump Width (SJW) on each recessive-to-dominant edge, keeping the sample point aligned with the transmitter. Only one synchronization is permitted within a single bit time (REQ-026). The two resynchronization cases are illustrated in Figure 6.

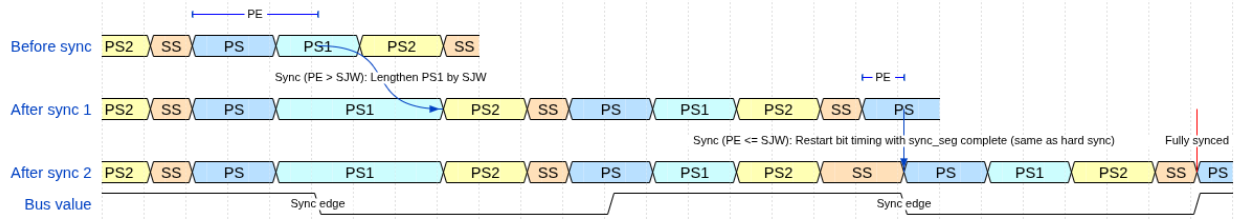

Figure 6: Resynchronization over two successive sync edges. PE is the phase error relative to SYNC_SEG. SJW is the Synchronization Jump Width.

CAN FD and the flexible data rate. CAN FD introduces a second, independently configured bit rate for the data phase. The BRS (Bit Rate Switch) bit in the FD control field (see Figure 4) controls this transition: when BRS is recessive, the bus switches to the data-phase bit rate immediately after the BRS sample point and returns to the nominal rate at the CRC delimiter. The nominal rate governs the arbitration phase (SOF through BRS) and the return path (CRC delimiter onward). The data rate governs the payload and CRC fields in between. Because the data phase operates at a much shorter bit time, the same physical propagation delay represents a larger fraction of the bit period. On electrically long buses at high data rates, the loop propagation delay can exceed a full data-phase bit time.

Transmitter Delay Compensation (TDC) addresses this. A transmitter in the FD data phase cannot rely on immediate bus loopback for bit-error monitoring, because the echo of a driven bit arrives one or more bit times late. TDC measures the actual round-trip delay at the start of the data phase and configures a Secondary Sample Point (SSP) at the correct offset, so that each transmitted bit is still checked for loopback correctness. The TDC measurement and SSP configuration are PCS responsibilities and are a significant driver of PCS complexity in the implementation (Section 7.7).

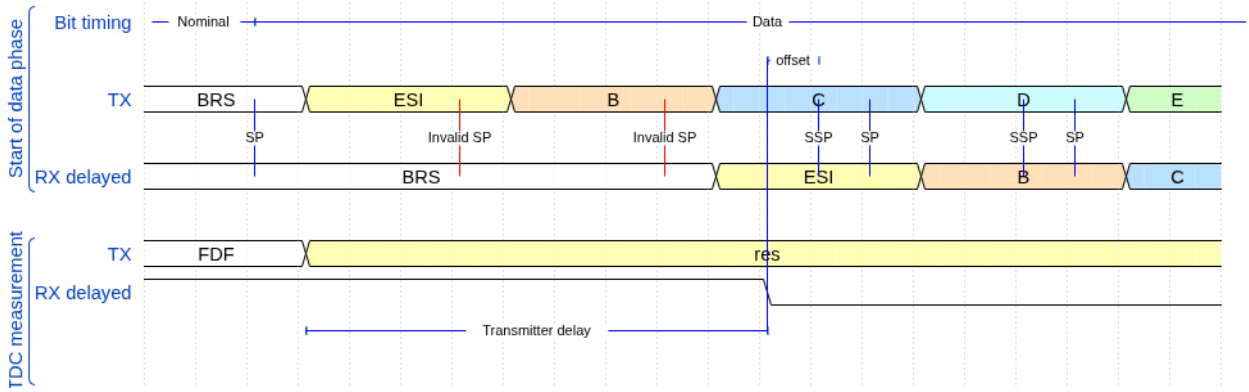

Figure 7: Transmitter Delay Compensation (TDC). The loop delay t_{loop} is measured on the first data-phase bit and used to position the SSP at $t_{\text{SSP}} = t_{\text{TDC\_offset}} + t_{\text{measured}}$.

4.4 Bit Stuffing

Bit stuffing ensures sufficient transitions on the bus for receiver clock synchronization. CAN Classic applies dynamic stuffing throughout the frame: after five consecutive bits of the same polarity, the transmitter inserts one complement stuff bit and the receiver removes it before forwarding the data stream (REQ-018). CAN FD retains dynamic stuffing through the arbitration phase, then switches to a combined dynamic-plus-fixed scheme in the data phase. Fixed stuff bits are inserted at predetermined positions (every fourth bit in the CRC field, independent of the preceding bit pattern). They carry a parity-encoded Stuff Bit Count (SBC) field that allows receivers to independently verify the number of dynamic stuff bits seen in the frame - an additional error detection layer absent in CAN Classic (REQ-016, Figure 8).

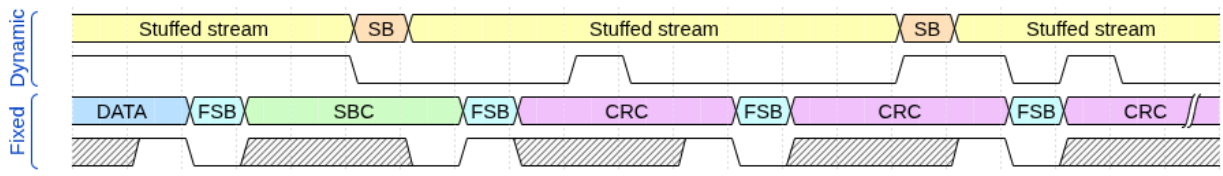

Figure 8: Dynamic and fixed bit-stuffing examples showing stuff bit placement for both encoding modes. The waveform below each row shows the resulting bus signal.

4.5 Cyclic Redundancy Check

The CRC polynomial and field length depend on frame type and data payload length (REQ-006):

- **CRC-15**: used for all CAN Classic frames. Dynamic stuff bits are excluded from the CC CRC computation - the CRC accumulates over the destuffed bit stream from SOF through the end of the data field (REQ-013).
- **CRC-17**: used for FD frames with data payloads up to 16 bytes (DLC 0-10).
- **CRC-21**: used for FD frames with data payloads from 20 to 64 bytes (DLC 11-15).

The key asymmetry between CC and FD concerns the CRC data feed. For FD frames, dynamic stuff bits up to and including the data field are included in the CRC computation, along with the SBC field itself. Fixed stuff bits are excluded from the CRC computation in both CC and FD frames (REQ-013). This dual data-feed requirement is a direct consequence of REQ-013 and has concrete consequences for the MAC implementation described in Section 7.5. The CRC field is terminated by a recessive CRC delimiter bit (REQ-017). A received frame with a CRC mismatch causes the detecting node to transmit an error flag.

4.6 Error Detection and Fault Confinement

Every CAN node monitors the bus for five categories of error (REQ-021). Bit errors occur when a transmitter reads back a polarity different from what it drove - with two exceptions: a recessive non-stuff bit overridden during arbitration (enabling multi-master bus access) and a recessive bit in the ACK slot (enabling receiver acknowledgment). Stuff errors occur when six consecutive bits of the same polarity appear where the stuffing rule prohibits it. CRC errors occur when the received checksum does not match the locally recomputed value. Form errors occur when fixed-format fields contain illegal bit values. Acknowledgment errors occur when a transmitter receives no dominant ACK bit from any receiver (REQ-014). Detection of any of these errors causes the detecting node to immediately transmit an error flag, aborting the in-progress frame (REQ-022). An error active node transmits an active error flag consisting of six consecutive dominant bits. An error passive node transmits a passive error flag of six consecutive recessive bits instead (REQ-007). Both are followed by an eight-bit recessive error delimiter.

The FCE tracks each node's error history through TEC and REC. Counter increments and decrements follow the rules in ISO 11898-1 sec. 8.1.4.2 (REQ-029). A node begins in error active and transitions to error passive when either counter exceeds 127 (REQ-030), then to bus off when TEC exceeds 255. In bus off the node ceases all bus activity and shall not influence the bus (REQ-027) until 128 sequences of 11 consecutive recessive bits are observed, after which TEC and REC are reset and the node returns to error active (REQ-030). A host-initiated `llc_i.normal_mode` assertion also resets the FCE to its initial state immediately (REQ-028). Whether error signaling is enabled at all is a run-time configuration parameter (REQ-036). This escalation mechanism is the subject of several verification plan requirements and directly motivates the separation of the FCE into a dedicated module with its own testbench.

With those mechanisms established - sub-layer boundaries, frame formats, bit timing and the dual data rate, stuffing rules, CRC polynomials, and the fault confinement escalation model - Section 5 introduces the five classification dimensions of the verification plan and shows how each one connects back to the protocol concepts described here.

5 Verification Plan

The requirements set established what must be true about the implementation - 38 entries, each naming a protocol obligation and linking it to its ISO clause. But requirements in that form are not yet actionable as verification tasks: they say nothing about which module testbench should exercise them, what stimulus configurations are needed, whether internal signals must be observable, or how completion will be recognized. Turning the requirements set into a verification plan means answering those questions explicitly for each entry, before implementation begins.

The plan was populated through the same Model Context Protocol server introduced in Section 3, which validated each field value against the schema before committing. The five classification dimensions fall into two groups. Three are design-facing - `layer`, `side`, and `format_applicability` - determining where each requirement belongs in the module decomposition and what stimulus configurations its testbench needs. Two are verification-facing - `observability` and `verification_method` - resolving whether a requirement can be checked through port signals or requires access to internal state, and specifying the verification technique. Priority spans both groups, driving implementation sequencing and determining which requirements must be closed before the design is considered complete. The rationale and allowed values for each dimension are described in Section 5.1 through Section 5.7.3, followed by the full verification plan data structure and its traceability fields in Section 5.7.

5.1 Layer

The layer field assigns each requirement to the protocol sub-layer that owns it (LLC, MAC, PCS, or FCE - see Section 4.1), determining the verification boundary at which the requirement must be exercised. A fifth label - **system** - classifies requirements that are inherently multi-layer or multi-node in character. Some CAN behaviors cannot be attributed to a single layer of a single node: they emerge from interactions between multiple nodes on the bus, or span the layer boundary within a single node. The system label flags these requirements as ones that require either an integrated multi-module testbench or a multi-node simulation environment.

At design time, this classification directly motivated the layered module architecture: requirements assigned to a given layer pointed to the corresponding module as the responsible implementation unit and to that module's testbench as the primary verification environment. The consequence of the system label is described further in Section 6.2.

5.2 Side

The side field records whether a requirement pertains to the transmitter path, the receiver path, or both roles simultaneously. This dimension reflects the ISO standard's own framing, which frequently specifies transmitter and receiver obligations separately. In the verification environment, the side field determines whether a testbench drives the DUT in transmitter mode, receiver mode, or both roles in succession within a single test scenario. The design consequences of this dimension - and why it appeared to motivate a split-path architecture but did not - are discussed in Section 6.2.

5.3 Format Applicability

The format_applicability field records which of the in-scope frame formats (CB, CE, FB, FE - see Figure 4, where CB and CE implicitly cover remote frame variants) each requirement applies to. Because the formats differ in stuffing mode, CRC polynomial, and control field structure (Section 4), a requirement that applies

only to FD frames implies stimulus configurations with FDF=1 and DLC values spanning both the CRC-17 and CRC-21 threshold, while a requirement that applies to all four formats must be exercised across all format-specific configurations. The field makes those implications explicit rather than leaving them to be inferred from the requirement text.

5.4 Observability

The observability field resolves each requirement as either black-box or white-box, relative to the module boundary of the owning layer:

- **Black-box**: Can be verified purely through the module's observable port signals.
- **White-box**: Verification requires direct observation of the module's internal state.

This distinction has direct consequences for testbench architecture. Black-box requirements are verifiable with stimulus-and-observe testbenches that drive inputs and check outputs without any knowledge of internal implementation. White-box requirements - which include CRC polynomial correctness, bit counter arithmetic, error counter thresholds, and Gray-coded SBC encoding - require a parallel reference model that re-computes the expected value independently, or direct observation of internal signals via testbench signal access.

5.5 Priority

The priority field classifies each requirement into one of three levels:

- P1 requirements are need-to-have - they must be verified before the design can be considered complete.
- P2 requirements are nice-to-have - they are verified in the normal verification cycle but do not block closure.
- P3 requirements are optional - addressed only if schedule permits.

The final plan contains 32 P1, four P2, and two P3 requirements. The demotion rationale for each requirement not rated P1 is given in Table 3. Of the 32 P1 requirements, 25 are closed. Five (REQ-001, REQ-003, REQ-005, REQ-032, REQ-035) remain not started pending `can_llc` implementation, REQ-021 is in progress with partial simulation coverage, and REQ-034 is in progress with sub-claim 2 verified but sub-claim 1 open. P2 and P3 requirements are addressed as schedule permits.

5.6 Requirement Distribution

Table 6 shows the 38 requirements distributed across layer and observability. MAC dominates both in count and in white-box density, reflecting the breadth of frame-encoding logic that must be verified against internal bit-level state. FCE requirements are entirely black-box: fault-confinement state transitions are fully observable through the node's error-state output signals without needing access to internal counters. System requirements - those that require a multi-node or multi-module environment - split roughly evenly between the two observability classes.

Table 6: Requirement distribution by layer and observability.

Layer	Black-box	White-box	Total
MAC	3	16	19
LLC	4	3	7

Layer	Black-box	White-box	Total
System	2	3	5
PCS	1	3	4
FCE	3	0	3
Total	**13**	**25**	**38**

5.7 Verification Plan Data Structure

The verification plan data structure (Table 7) augments each requirement with the dimensions needed to answer not just *what* must be true, but *how* it will be verified, *where* the evidence lives, and *when* verification is complete. The plan evolved continuously as implementation and verification work progressed. Two dimensions were not part of the initial taxonomy: the `system` layer label was added when it became clear that some CAN behaviors emerge from multi-node interactions and cannot be attributed to any single module's testbench. The `observability` field was introduced when the distinction between black-box and white-box verification had direct consequences for testbench architecture that were not apparent from the requirement text alone. The following sub-sections detail the rationale and allowed values for each field in the verification plan. The complete verification plan is reproduced in Section C as two separate tables (linked by common IDs).

Table 7: Verification-plan data structure fields.

Field	Purpose
`id`	Sequential identifier REQ-NNN.
`source_clause`	ISO 11898-1:2015 section reference.
`original_wording`	Verbatim normative text excerpts from the ISO standard.
`paraphrase`	Concise paraphrase of the `original_wording` field (this is the actual requirement)
`layer`	Sub-layer owner: LLC, MAC, PCS, FCE, or system (Section 5.1).
`side`	transmitter, receiver, or both (Section 5.2).
`format_applicabilit`	Applicable frame formats: CB, CE, FB, FE (Section 5.3).
`observability`	`black_box` or `white_box` (Section 5.4).
`verification_methoc`	Method(s) used to verify the requirement (Section 5.7.1).
`priority`	P1 (need-to-have), P2 (nice-to-have), or P3 (optional) (Section 5.5).
`status`	`not_started`, `in_progress` or `complete` (Section 5.7.3).
`notes`	Residual clarifications not resolved by the paraphrase - implementation constraints, out-of-scope markers, or known ambiguities flagged for design review.
`label`	Assertion label, TB procedure name, coverage ID, or RTL tag. Comma-separated when multiple procedures cover distinct sub-claims (Section 5.7.2).
`file`	Target file: TB for simulation/coverage, RTL for code inspection. Comma-separated when sub-claims span multiple files (Section 5.7.2).

5.7.1 Verification Method

The verification_method field makes the path from requirement to verification artifact explicit and actionable. Four methods are used: `simulation` (automated assertion procedures in a testbench), `code_inspection` (RTL source review), `waveform_inspection` (manual review of simulation output),

and `coverage` (a functional coverage bin that records whether a specific condition or value range was exercised during simulation). Combinations are valid when multiple sub-claims within one requirement each call for a different method.

5.7.2 Traceability: Label and File

Each requirement entry carries two dedicated traceability fields: a `file` field identifying the testbench or RTL source file responsible for covering the requirement, and a `label` field identifying a specific named procedure, assertion, or coverage ID within that file. Together they establish a direct, navigable link from each requirement to its verification artifact.

5.7.3 Status

The status field (`not_started`, `in_progress`, `complete`) records requirement closure state explicitly, allowing partial progress to be tracked.

The verification plan - 38 requirements each classified along five dimensions and each linked to a testbench file and assertion label - served both roles in what followed. The design-facing dimensions (layer, side, format_applicability) constituted the primary architectural inputs for the design phase, mapping requirements to module boundaries and implementation scope. The verification-facing dimensions (observability, verification_method, label, file) defined the testbench architecture and evidence type for each requirement. How the design-facing dimensions shaped the module decomposition - and where the apparent mapping from requirements structure to design structure broke down - is the subject of Section 6.

6 Design and Architecture

The verification plan classified all 38 requirements along three design-facing dimensions - `layer`, `side`, and `format_applicability`. Two led directly to sound architectural choices. One pointed toward a split TX/RX architecture that was attempted but created more problems than it solved, and was replaced by the unified `can_mac_fsm`.

6.1 Adopting the ISO 11898-1 Sub-layer Model

The `layer` dimension of the verification plan assigns every requirement to a specific ISO 11898-1 sub-layer. Mapping each sub-layer to a dedicated module is therefore the natural decomposition. Module boundaries align directly with verification targets, each requirement points unambiguously to the responsible implementation unit, and each module can be exercised in isolation without driving frame-level stimulus through unrelated sub-layers.

6.2 Combined vs. Separated TX/RX Paths

The `side` dimension classifies each requirement as TX-only, RX-only, or both - making separate TX and RX paths appear natural, since each could be exercised independently. In practice the split created more problems than it solved.

A split path duplicates both code and hardware. Shared protocol logic must exist in both paths, and each path needs its own bit stuffer and CRC engine - putting two instances of `can_mac_bs` and `can_mac_crc` on the device where one of each suffices. As shown in Figure 11, the unified design shares a single instance of each.

A further cost is implementation and debugging complexity. Developing the two paths sequentially, solutions to shared protocol logic tended to diverge between implementations with no protocol justification. Debugging similarly doubles the investigation surface - tracing a bug requires reading two independent sets of waveforms.

The unified TX/RX path - a single `can_mac_fsm` with shared `can_mac_bs` and `can_mac_crc` instances - eliminates all three costs. Shared protocol logic exists once, preventing implementation drift. Debugging involves a single FSM and a single set of waveforms.

6.3 Per-Field FSM Granularity

The `format_applicability` dimension of the verification plan expresses requirements at the level of individual protocol fields. A requirement applies to a specific field in a specific set of frame formats. The natural FSM granularity is therefore one state per protocol field. With this structure, format-dependent transitions become state graph edges rather than counter conditionals inside a shared state, each requirement maps directly to a named FSM state, and testbench assertions can reference states rather than bit-count ranges. The detailed state graph is described in Section 7.2.

6.4 LLC Frame Format

6.4.1 Host-LLC Interface Format

All six bus frame types (CB, CE, FB, FE data frames, and remote frames for CB and CE) are represented at the host-LLC interface using the 71-byte LLC frame format shown in Figure 9. The layout extends the existing CAN Classic controller's LLC frame format, with bytes 0-3 carrying the identifier, byte 4 carrying frame type and DLC, and data beginning at byte 5 - matching the field positions used by the prior implementation. The extension adds 56 additional data bytes (bytes 5-68, zero-padded to 64 bytes) and two trailing flag bytes (bytes 69-70) carrying IDE, BRS, ESI, and RTR, so host software requires no change to the fields it already uses.

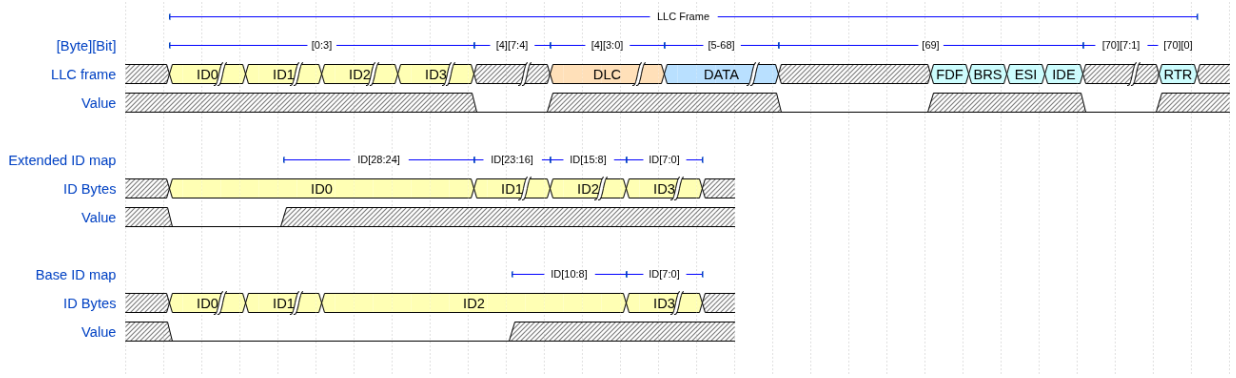

Figure 9: LLC frame format (71 bytes) at the host-LLC interface, with identifier byte mapping for base and extended IDs.

6.4.2 Internal LLC Frame Format

The host-LLC format places all control flags at the end of the frame. FDF, BRS, and ESI occupy byte 69, and IDE and RTR occupy byte 70 - after up to 64 bytes of payload. A serializer that consumed this stream in field order would need to buffer the entire 71-byte frame before it could begin transmitting, because IDE determines how many ID bits to drive (11 or 29), FDF determines which CRC polynomial and stuffing

mode to use, and BRS determines whether to signal the PCS to switch bit rate at the BRS boundary. This buffering requirement conflicts with the streaming architecture.

The design avoids this by defining a separate internal format for the MAC-facing stream, shown in Figure 10. All frame metadata is packed into two leading config bytes, followed by the ID and data bytes. With this layout, `can_mac_ser` extracts all frame metadata after receiving just two bytes and can begin streaming ID bits from the third byte onward. No frame buffering is needed - the MAC FSM receives each metadata field before it is required in the frame field sequence.

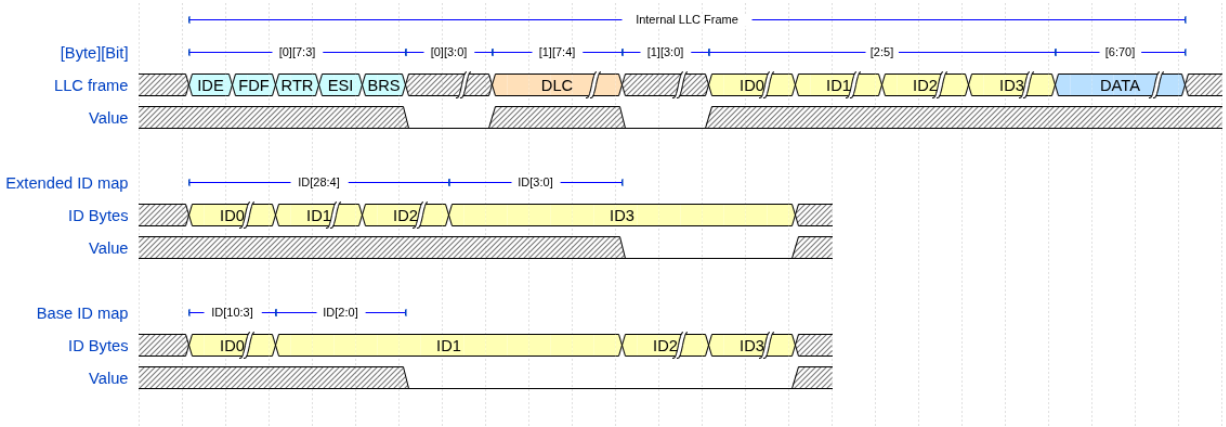

Figure 10: Internal LLC frame format at the `can_mac_ser` input, with identifier byte mapping for base and extended IDs.

6.5 System Overview

Figure 11 shows the complete module decomposition. The primary data path runs from `can_llc` through `can_mac` to `can_pcs`. The LLC receives frames from the host over an Avalon-ST interface and streams them byte-by-byte to the MAC serializer. The MAC FSM drives the serialized bit stream to the PCS, which applies bit timing and produces the sample-point and SSP strobes that the MAC uses to read and write the bus. `can_fce` sits outside the primary data path, receiving error and success events from the MAC and feeding node-state signals (error active, bus off) back to both the MAC and PCS. The PCS additionally pulses `can_fce` with idle conditions to drive bus-off recovery. A centralized types package (`pk_can_types`) defines all protocol constants, interface records, and reset values shared across modules.

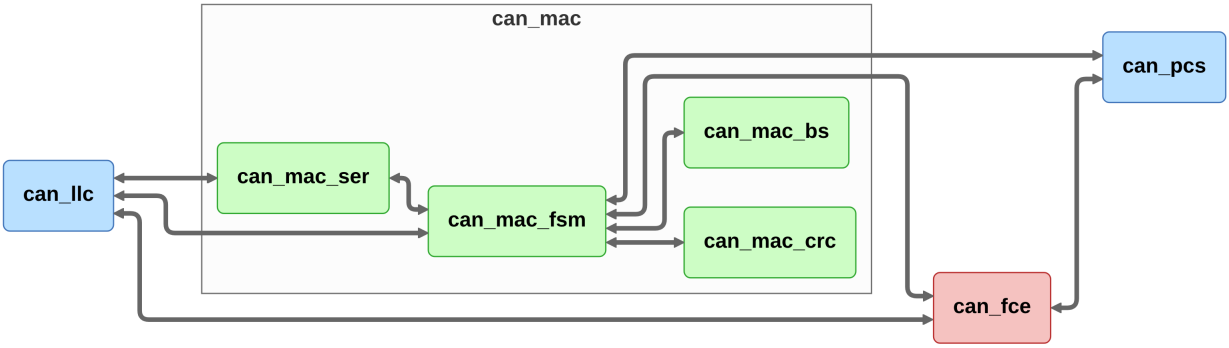

Figure 11: Implementation module decomposition showing the five entities and their inter-module connections.

7 Implementation

Section 6 concluded that a unified `can_mac_fsm` - one FSM, one bit stuffer, one CRC engine - was the right architecture once the split-path approach was attempted and found unworkable. This section shows what that architecture looks like in practice. The implementation decisions not derivable from the requirements table alone but forced by protocol structure include: the bit stuffer's handling of a pending dynamic stuff bit at the dynamic-to-fixed mode boundary, the CRC engine's combinatorial output mux and why a registered stage would violate the ISO IPT constraint at minimum prescaler, and the PCS synchronization rules that the prior implementation violated. A second set of observations shows where the unified FSM pays off - where a protocol rule that applies equally to transmitter and receiver is expressed once in shared state rather than twice in parallel FSMs. Subsections are ordered MAC-first: the FSM and its internal submodules (`can_mac_ser`, `can_mac_bs`, `can_mac_crc`) are described before the supporting `can_fce` and `can_pcs` layers, so that the FSM's interface contracts are established before the modules that satisfy them. `can_llc` is not yet implemented. Its interface contracts are captured in the verification plan (REQ-001 through REQ-005, REQ-032).

7.1 Interface Conventions

Everllence's `std_logic`-only port constraint does not preclude the use of VHDL record types on entity ports - records of `std_logic` and `std_logic_vector` fields satisfy the constraint. The design uses typed record interfaces (e.g., `t_can_mac_pcs_if_m2s`, `t_can_mac_fsm_bs_if_s2m`) to bundle related signals into a single port. Each record type has a corresponding reset constant (e.g., `c_can_mac_pcs_if_m2s_reset`), ensuring that every module can be reset to a known state without manually enumerating each field.

The alternative - flat port lists with individual `std_logic` signals - was used in the existing controller, where the FSM entity has 20 individual ports for its various submodule interfaces. This approach becomes unwieldy as inter-module signal counts grow: bundling related signals into records reduces each entity's port list to a handful of typed ports, keeping the interface manageable as the design scales.

The naming convention follows the data-flow direction: `m2s`/`s2m` (master-to-slave/slave-to-master) for control interfaces, and `s2d`/`d2s` (source-to-destination/destination-to-source) for Avalon-ST data-transfer interfaces. This convention is consistent with Everllence's existing interface naming and makes the direction of data flow explicit at every port.

7.2 `can_mac_fsm`

The `can_mac` sub-layer is built around a single unified FSM entity (`can_mac_fsm`). The complete signal-level interface is reproduced in Section B.

7.2.1 FSM Structure and Mode Flag

`can_mac_fsm` contains two synchronous processes (`p_fsm` and `p_stream_to_LLC`) and one `t_fsm_state` enum covering 19 states. An `is_transmitter` boolean signal is latched to `true` when the FSM drives the SOF dominant bit at the start of a new frame transmission and cleared at arbitration loss or at the end of the EOF field. Once latched, `is_transmitter` remains stable for the rest of the frame, partitioning per-state logic into a TX branch (`drive_bit` strobe: determines and drives `pcs_o.tx_data`) and an RX branch (`sample_point` strobe: advances state and captures received bits). The state transitions at the sample point are shared between TX and RX in most states. Only `s_ack` and `s_eof` carry role-specific logic on

the sample-point path (ACK success latching, receiver dominant assertion, and frame-completion signaling respectively).

The per-field state granularity introduced in Section 6.3 is preserved in the unified FSM. The arbitration region uses two states: SOF is driven in `s_bus_idle`, and the ID bits, RTR/SRR/RRS, and IDE share `s_arbitration` with `bit_count` indexing into the 32-bit ID field. Each post-arbitration protocol field (FDF, RES, BRS, ESI, DLC, Data, SBC, CRC, CRC Delimiter, ACK, ACK Delimiter, EOF) has a dedicated state, making field boundaries explicit in the state encoding. The complete FSM is shown in Figure 12.

7.2.2 TX Mode: Frame Transmission

When `is_transmitter = true`, the FSM drives bits at each bit-boundary strobe and monitors the bus echo at each sample point to detect bit errors, ACK, and arbitration loss. In the CAN FD data phase, the SSP strobe replaces the SP for bit-error monitoring. The MAC compares `pcs_i.rx_data` against `transmitted_bits_shift_reg(tdc_delay)` - where `tdc_delay` is supplied alongside the SSP strobe by the PCS - to check the bit that was transmitted `tdc_delay` bit times earlier [8, Sec. 7.3.4]. A register depth of 8 provides margin for realistic transceiver loop delays and data-phase bit rates. Arbitration loss clears `is_transmitter` in-place in `s_arbitration`, which then continues as an RX observer.

The CRC and bit-stuffer feed source requires special handling at the arbitration boundary. Throughout `s_arbitration` both transmitter and receiver feed `pcs_i.rx_data` - the actual bus value - rather than the locally driven bit. If the transmitter loses arbitration and continues as a receiver, its CRC accumulator must hold exactly the value a pure receiver would have built. From `s_fdf_r1_r0` onward the transmitter switches to `transmitted_bits_shift_reg(0)`, which avoids any dependency on `pcs_i.rx_data` echo latency once TDC is active.

7.2.3 RX Mode: Frame Reception

When `is_transmitter = false`, the FSM observes `pcs_i.rx_data` at each sample-point strobe and stores received bits directly into an internal `llc_frame` byte array. No separate deserializer is needed. The bit stuffer is driven from `pcs_i.rx_data` to perform destuffing, and the CRC engine accumulates the received bit stream in parallel. The FSM validates the SBC field (FD frames), compares the received CRC against the locally accumulated result, and checks form bits (reserved bits, CRC delimiter, ACK delimiter, EOF) for required polarities. A mismatch in any of these fields triggers a transition to the error-frame sequence. During the ACK slot the FSM drives `pcs_o.tx_data = c_dominant` for one bit (`bit_count = 0`) regardless of frame format. The FD ACK slot spans two bits but the receiver asserts dominant only during the first. The bus is released after the ACK delimiter [8, Sec. 8.1.4.2.b].

During `s_intermission`, the completed frame is streamed byte-by-byte to the LLC RX sink over the Avalon-ST interface, and successful reception is signaled to the FCE. This design eliminates the need for a separate deserializer entity on the RX path - the frame buffer is populated and streamed entirely within `can_mac_fsm`.

7.2.4 Error-Frame States

The FSM uses two explicit error-frame states. `s_error_flag` drives the 6-bit flag and `s_error_delimiter` counts the 8-bit recessive delimiter. The delimiter state has two sub-phases: first awaiting the bus to go recessive (other nodes may still be driving their own flags), then counting the remaining recessive bits. A dominant during the delimiter restarts the error-frame sequence - either as a new error or as an overload

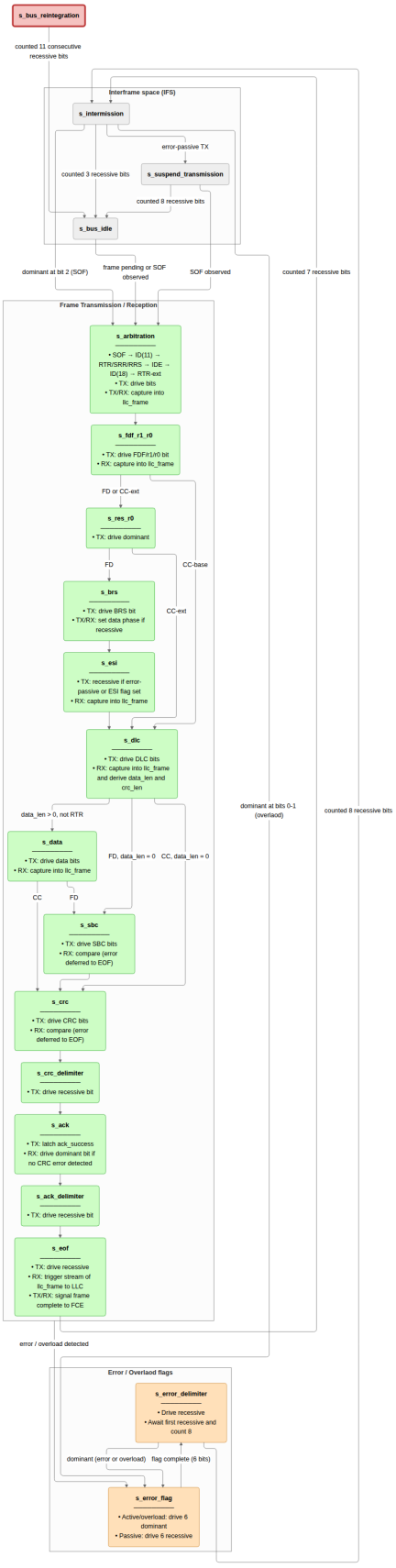

Figure 12: `can_mac_fsm` (19 states) controlling TX and RX for all in-scope frame formats. Arbitration loss clears `is_transmitter` in place without a state transition.

condition on the last delimiter bit [8, Sec. 8.1.4.2.f]. Both transmitter and receiver errors enter this same two-state sequence: the flag polarity is `not fce_i.error_active`, determined by the FCE node state rather than the TX/RX role, and `s_error_delimiter` is entirely role-independent.

7.3 `can_mac_ser`

`can_mac_ser` converts the LLC byte stream into a serial polarity bit stream for the MAC FSM. Its four-state FSM manages the two-byte configuration handshake, byte fetching, and bit-by-bit serialization. The serializer extracts LLC metadata (IDE, FDF, DLC, FTYP, BRS, ESI) from the two config bytes and registers it in `t_llc_metadata`, which remains stable for the entire frame. The internal frame format that makes this possible is described in Section 6.4.2. The serializer forwards `transfer_status` from the FSM back to the LLC, returning to `s_load_config_byte_0` on any non-ongoing status so that errors and aborts terminate serialization immediately.

The 32-bit ID field in the internal format is left-aligned: a base identifier (11 bits) occupies bits [31:21], leaving 21 unused bits at the LSB end. An extended identifier (29 bits) occupies bits [31:3], leaving 3 unused bits. The serializer tracks this with two counters initialized in `s_load_config_byte_1` from the `ide` flag: `id_bits_remaining` counts real ID bits still to be presented, and `padding_bits_remaining` counts trailing unused bits to be skipped. In `s_shift_out_bits`, padding bits are advanced without asserting `valid`, so the MAC FSM never observes them. This means the FSM always receives exactly 11 or 29 consecutive valid ID bits regardless of frame format, with no format-specific logic required downstream. The four-state serializer FSM is shown in Figure 13.

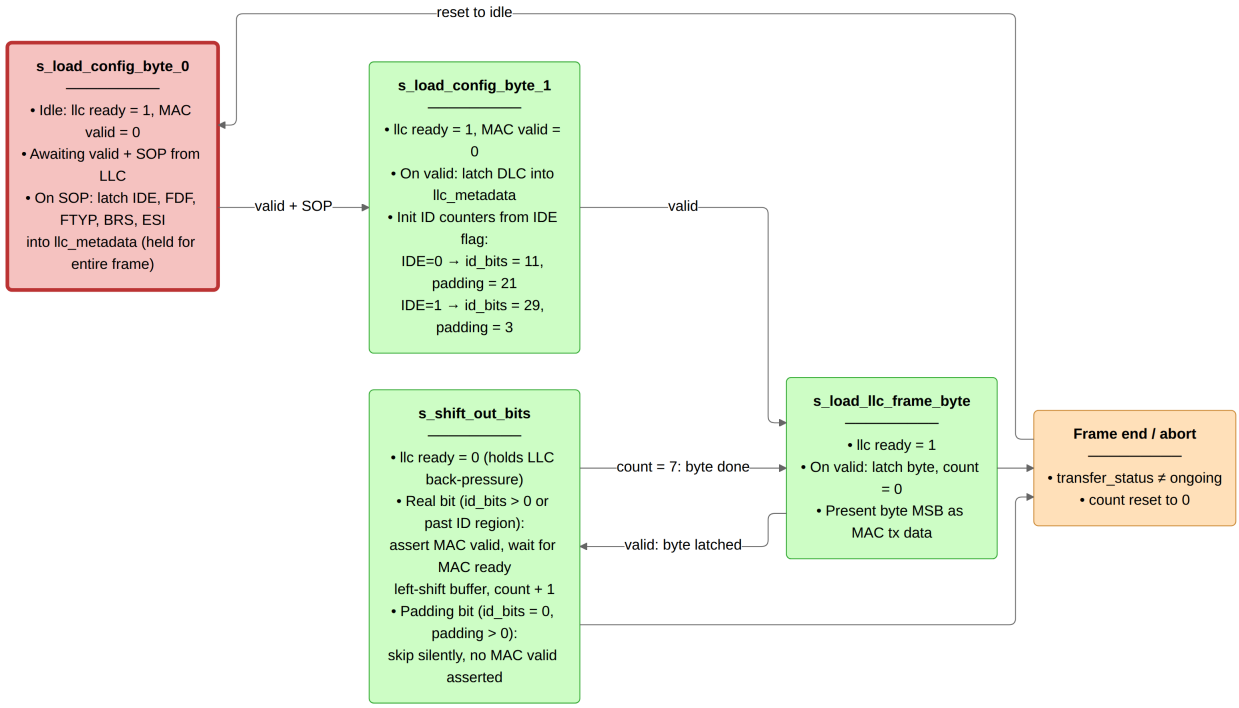

Figure 13: `can_mac_ser` FSM (four states) serializing the internal LLC frame to the MAC bit stream. Unused padding bits in the 32-bit ID field are skipped silently.

The bit stream it produces feeds the bit stuffer, which may insert additional stuff bits before the FSM drives each bit to the PCS interface.

7.4 can_mac_bs

can_mac_bs implements both dynamic and fixed bit stuffing for CAN Classic and CAN FD frames [8, Sec. 10.6]. The single entity is instantiated once inside can_mac_fsm and serves both TX stuffing and RX destuffing via the same logic - the FSM drives the same bs_i interface regardless of role, and the stuffer's output is either inserted into the TX bit stream or used by the FSM to discard a received destuff bit.

In **dynamic mode** (fixed_bit_stuffing_en = '0'), the stuffer counts consecutive bits of identical polarity and emits an inverse-polarity stuff bit after every five (REQ-018). A binary stuff_count counter is incremented on each dynamic stuff bit. Its value is Gray-coded and parity-encoded into the stuff_bit_count output, which the FSM reads when transmitting the SBC field (REQ-016).

In **fixed mode** (fixed_bit_stuffing_en = '1'), used for the FD CRC region, one fixed stuff bit (FSB) is emitted immediately on the rising edge of fixed_bit_stuffing_en, then one FSB every four real bits (REQ-018). The FSB polarity is always the inverse of the preceding bit, so a receiver can detect a form error if the FSB matches its predecessor.

The transition from dynamic to fixed stuffing requires special handling when a dynamic stuff bit is already pending at the rising edge of fixed_bit_stuffing_en. Suppressing the pending dynamic SB would cause a TX/RX divergence: the transmitter and receiver would derive different stuff_count values from the same bit stream, causing SBC mismatch. The implementation instead promotes the pending dynamic SB to the initial FSB - the two coincide and both requirements are satisfied simultaneously (see Figure 14, REQ-016, REQ-018). On the falling edge of fixed_bit_stuffing_en, any pending FSB is cancelled immediately. The MAC FSM exits fixed stuffing at the last CRC bit without providing a slot to drain a still-pending FSB.

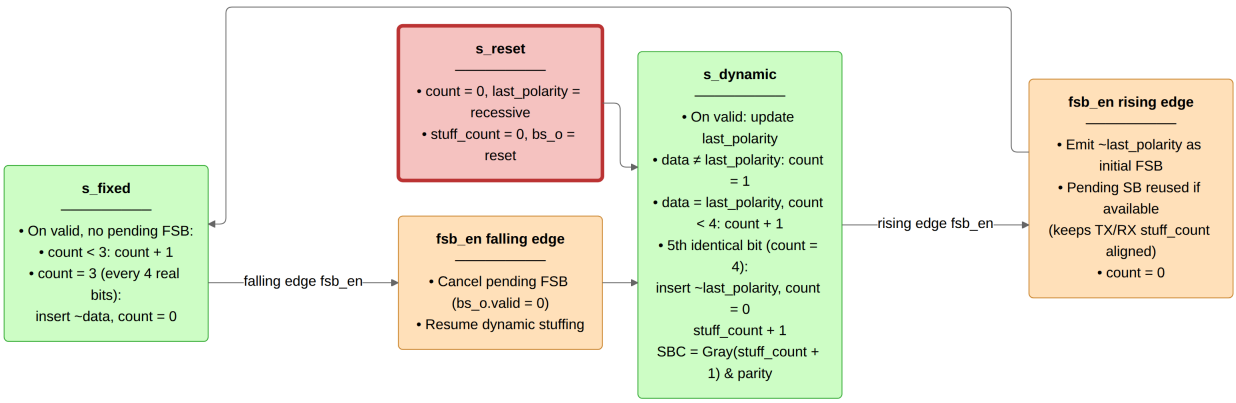

Figure 14: can_mac_bs operating in dynamic and fixed stuffing modes. A pending dynamic stuff bit at the rising edge of fsb_en is promoted to the initial FSB rather than suppressed.

Its stuff_bit_count output is the SBC value the FSM reads when transmitting the SBC field.

7.5 can_mac_crc

CAN Classic computes its CRC over the raw bit stream excluding stuff bits, while CAN FD includes dynamic stuff bits up to and including the data field - a difference that exists because the FD CRC must protect the stuff-bit count as well as the data. A single data feed to the CRC engine is therefore insufficient: CC and FD frames require different input streams. The design exposes two feeds on the CRC interface (data_cc and data_fd) so the FSM can drive both simultaneously and the CRC module requires no protocol knowledge about which stream to select.

`can_mac_crc` provides CRC generation and checking for both CAN Classic and CAN FD frames. CAN Classic frames use CRC-15, while CAN FD frames use CRC-17 (data payloads up to 16 bytes) or CRC-21 (data payloads above 16 bytes) [8, Sec. 10.4.2.6]. The single entity is instantiated once inside `can_mac_fsm`, serving both TX (generation) and RX (checking). The FSM sets `crc_poly_select` from the DLC field in `llc_metadata` before the first frame bit is driven: because the internal LLC frame format (Section 6.4.2) delivers DLC in config byte 1, the polynomial is known upfront and requires no mid-frame switching.

Three parallel `gen_crc` instances run continuously on separate data feeds: `data_cc` drives CRC-15 via `valid_cc`, while `data_fd` drives both CRC-17 and CRC-21 via `valid_fd`. The output multiplexer selects the active engine's result based on `crc_poly_select` and left-aligns it to the common 21-bit output width by zero-extending the shorter results at the least significant bit: CRC-15 occupies bits [20:6], CRC-17 occupies bits [20:4], and CRC-21 occupies the full width.

The output mux (`p_crc_mux`) is combinatorial rather than registered. Each `gen_crc` instance registers its accumulator on the rising edge, so the mux selects over three curre registered values - the rationale for registering module outputs is satisfied one level down. Omitting the register keeps IPT at 2 system clocks: SP+1 for the final accumulator update, SP+2 for the mux read. ISO 11898-1 §7.3.3 mandates IPT ≤ 2 t_q (REQ-024), and at minimum prescaler (m=1, t_q = 1 system clock) that leaves exactly 2 system clocks. A registered mux would require 3 system clocks, violating this bound at m=1 (see Figure 15).

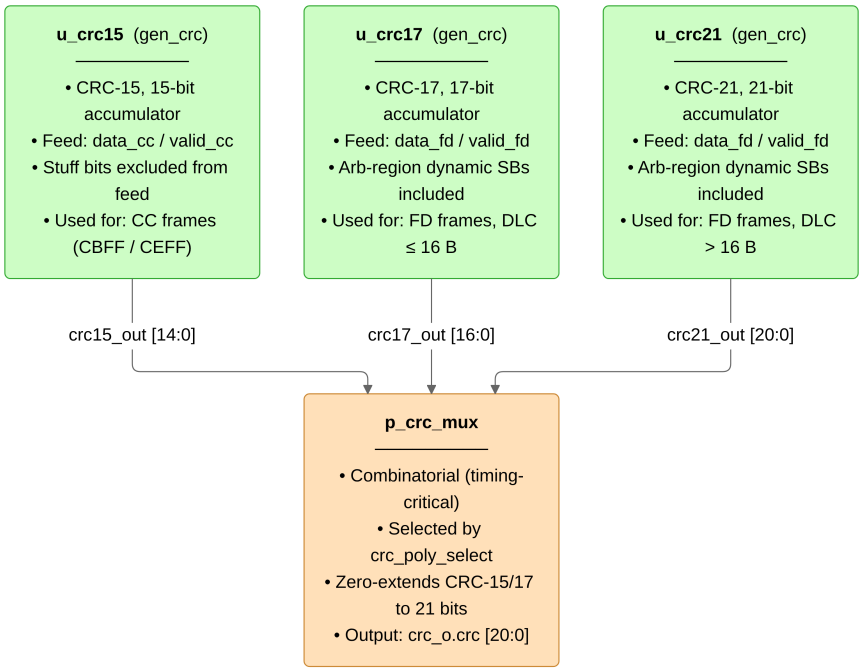

Figure 15: `can_mac_crc` dataflow with three parallel CRC engines. The output mux is combinatorial to satisfy the ISO IPT ≤ 2 t_q constraint at minimum prescaler.

7.6 `can_fce`

`can_fce` implements the error FSM and counter management specified in ISO 11898-1 [8, Secs. 8.1.3–8.1.4]. It maintains TEC (Transmitter Error Counter) and REC (Receiver Error Counter) and transitions between three states: `s_error_active` (normal operation), `s_error_passive` (TEC or REC > 127), and `s_bus_off` (TEC > 255), as shown in Figure 16.

Counter updates follow the rules in [8, Sec. 8.1.4.2]: TEC increments by 8 on TX errors, with

`mac_i.passive_tx_ack_error_exempt_1` suppressing the increment for the passive ACK error exemption (ISO 8.1.4.2.c, Exception 1). TEC decrements by 1 on successful TX. REC increments by 1 on RX errors during non-error-flag phases, by 8 on primary errors or error-flag-phase errors, and decrements by 1 or clamps to 127 on successful RX. Bus-off recovery requires counting 128 `pcs_i.idle_condition` pulses (11 consecutive recessive bits each) from the PCS, which resets both counters and returns the FSM to `s_error_active`. `llc_i.normal_mode` is a supervisory reset per ISO 11898-1 that forces the same transition from any node state immediately.

The one counter rule that requires careful reading of the ISO prose is the passive ACK error exemption (ISO 8.1.4.2.c, Exception 1): an error passive node that transmits a frame and receives no dominant ACK bit shall not increment TEC, because the node's passive error flag is recessive and may itself prevent receivers from asserting the ACK slot. The FCE has no frame-level visibility - it receives event signals from the MAC, not raw bus bits - so the MAC must explicitly signal this case via `mac_i.passive_tx_ack_error_exempt_1`, asserted when the FSM detects an ACK error while `mac_o.error_active` is deasserted. Without this signal the FCE would treat an unacknowledged passive-node transmission identically to any other ACK error and escalate TEC unnecessarily.

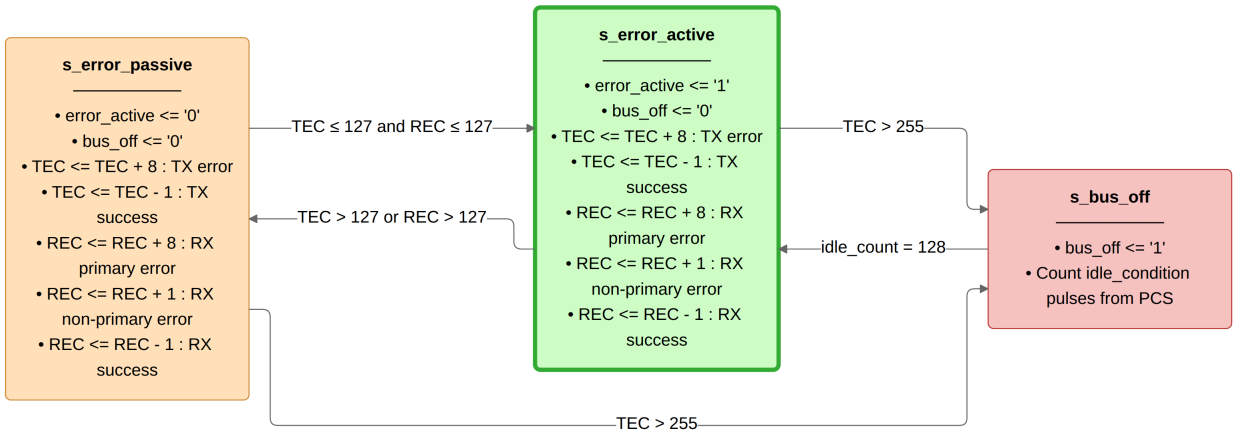

Figure 16: `can_fce` FSM governing the error active, error passive, and bus off node states per ISO 11898-1 sec. 8.1.4.4.

7.7 `can_pcs`

`can_pcs` is a cyclic bit-timing engine: its internal `t_segment` register advances through `s_sync_seg` (1 TQ, fixed), `s_prop_seg`, `s_phase_seg1`, and `s_phase_seg2` on every TQ boundary. The SP strobe and `rx_data` latch fire at the end of `s_phase_seg1`. The TX bit is driven at the end of `s_phase_seg2`. When `fce_i.bus_off` is asserted, the SP slot counts consecutive recessive bits and pulses `fce_o.idle_condition` every 11 bits for FCE bus-off recovery. The full timing operation is shown in Figure 17.

7.7.1 Resynchronization

`can_pcs` implements all four sub-claims of REQ-026. Sub-claim 1 (one synchronisation per bit time) is enforced by a `sync_applied` flag that gates the edge-qualify predicate and clears at each sample point. Sub-claim 2 (only R⟶D edges qualify, previous SP must be recessive, no sync while transmitting) is enforced by `sync_applied` gating sync to the post-SP window, where `rx_bus_prev` holds the SP-sampled value, making the R⟶D check simultaneously a check on SP polarity. A transmitter guard suppresses sync during

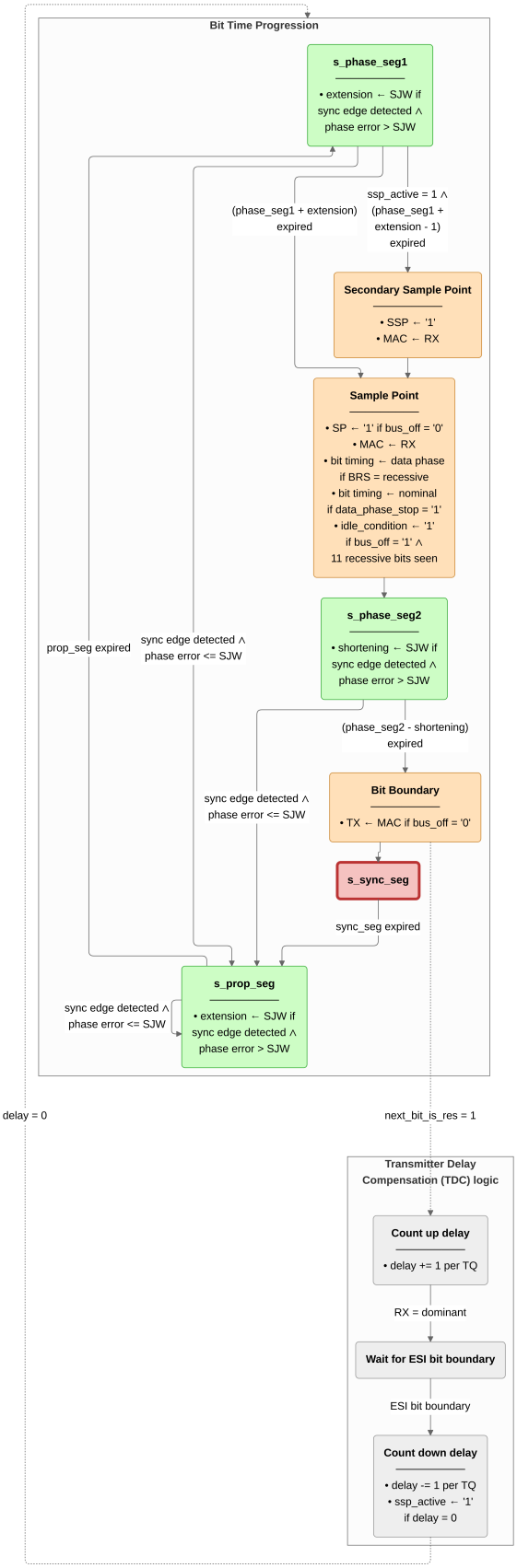

Figure 17: `can_pcs` bit-time FSM with concurrent resynchronization and TDC pipelines per ISO 11898-1 sec. 7.2-7.4.

TX. Sub-claim 3 (hard sync) is MAC-controlled via `do_hard_sync`, allowing the MAC to select hard sync at the FDF-to-res transition without disturbing PCS timing state. Sub-claim 4 (resync by segment adjustment) adjusts Phase_Seg1 or Phase_Seg2 by up to SJW. Errors not exceeding SJW produce the same effect as a hard synchronisation.

7.7.2 Dual Bit Rate Switching

`can_pcs` holds no frame-format knowledge. Rate switching is entirely MAC-driven through three dedicated control signals on the MAC-PCS interface.

At the BRS sample point, `can_pcs` reads BRS polarity and, if recessive, replaces the active segment lengths and SJW with their data-phase counterparts for the remainder of the data phase (see Figure 17). TDC measurement is armed at the FD reserved bit boundary (Section 7.7.3). `mac_i.data_phase_stop`, asserted by the MAC at the CRC delimiter SP or on error-frame entry, restores nominal timing and clears all TDC state.

This interface design keeps protocol knowledge in the MAC layer and timing knowledge in the PCS layer, following the ISO 11898-1 layered architecture. `can_pcs` does not inspect DLC or frame format - the only frame-level observation it makes is reading the BRS bit polarity when `mac_i.next_bit_is_brs` is set, making it independently testable against any timing configuration without frame-level stimulus.

7.7.3 Transmitter Delay Compensation

The motivation and principle of TDC are described in Section 4.3. The implementation uses a three-stage pipeline within `p_can_pcs`: count up from the FD reserved bit boundary until the TX-to-RX echo arrives, wait for the first data-phase bit boundary, then count down the measured delay to arm the SSP. Once armed, current_temp SSP fires at a fixed offset within `s_phase_seg1` every data-phase bit time. The measured round-trip delay is supplied to the MAC as `mac_o.tdc_delay` for indexing into the transmitted-bit shift register (Section 7.2). The full TDC pipeline is shown in Figure 17.

8 Verification and Results

The implementation described in Section 7 was exercised against the 38-requirement verification plan (Section 5) using five dedicated testbenches, each aligned to a module boundary established by the layered architecture. `can_mac_pcs_fce_tb` is the primary integration testbench, exercising two `can_mac_pcs_fce` instances connected through a dominant-wins bus model and covering 16 requirements spanning MAC frame encoding, PCS bus-off recovery, and FCE-driven error flag generation. The four unit testbenches - `can_mac_crc_tb`, `can_mac_bs_tb`, `can_pcs_tb`, and `can_fce_tb` - target individual submodules with focused stimulus. `can_mac_ser_tb` exercises the serializer but has no standalone requirement entries in the verification plan. Of the 38 requirements, 27 are closed. Table 8 lists the five testbenches and their coverage. Figure 18 shows the testbench architecture.

REQ-013 (dual CRC data feed: CC excludes dynamic stuff bits, FD includes them) and REQ-023 (overload frame conditions) are verified by code inspection against `can_mac_fsm.vhd` rather than simulation. `can_mac_crc_tb` closes REQ-006 via three coverage bins: CB and CE frames (CRC-15), FD frames with payload up to 16 bytes (CRC-17), and FD frames with payload greater than 16 bytes (CRC-21), all hit. `can_mac_bs_tb` closes REQ-016 via the `p_sbc_checker` assertion and REQ-018 via input and output coverage bins, all bins hit. `can_pcs_tb` closes REQ-025 via the `p_check_tdc_delay` assertion.

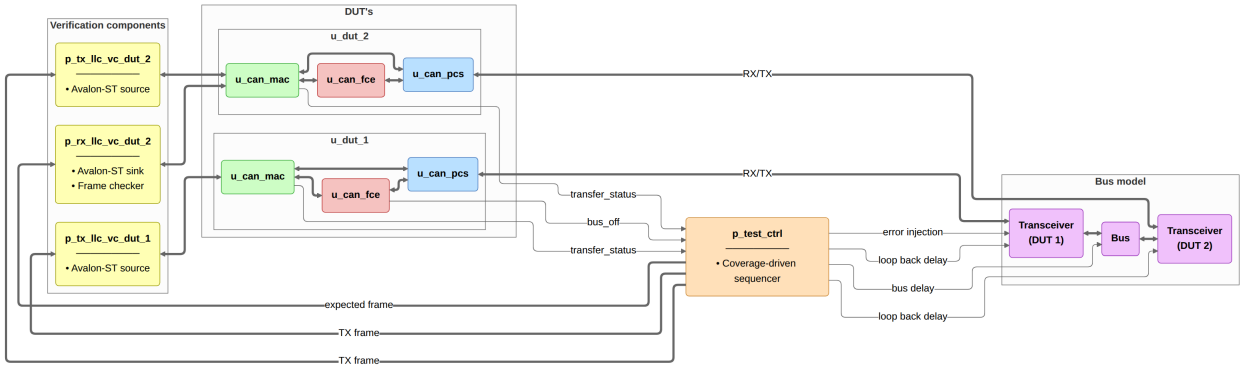

Figure 18: `can_mac_pcs_fce_tb` integration testbench. Two `can_mac_pcs_fce` instances connect through a dominant-wins bus model. Avalon-ST VCs drive and sample the MAC interfaces. `p_test_ctrl` sequences test stimuli, injects bit errors via `dut_1_rx_recessive`, and reads pass/fail status from the TX-status and bus-off monitors.

Figure 19 shows the two-node integration scenario from `can_mac_pcs_fce_tb`: a complete FD frame transmitted by DUT 1 and received by DUT 2, with SSP pulses confirming TDC is active during the data phase and resynchronization events visible on DUT 2 (REQ-010, REQ-012, REQ-014, REQ-017, REQ-026). Figure 20 shows the bit stuffer's dynamic and fixed mode behavior from `can_mac_pcs_fce_tb` (REQ-016, REQ-018). Figure 21 shows dual bit rate switching and TDC measurement from `can_mac_pcs_fce_tb`: the PCS replaces nominal segment lengths at the BRS sample point and positions the SSP once the transceiver loopback delay is measured (REQ-015, REQ-024, REQ-025, REQ-031). Figure 22 shows the arbitration loss scenario from `can_mac_pcs_fce_tb`: the losing node clears `is_transmitter` in-place at `s_arbitration` and continues as receiver without a state transition (REQ-020). Figure 23 shows the error frame escalation: a bit error on the SOF bit triggers the first error flag, and subsequent bit errors on each dominant error flag bit escalate TEC to 128, transitioning the node to error passive and inserting `s_suspend_transmission` after `s_intermission` (REQ-007, REQ-008, REQ-021, REQ-022, REQ-029). Figure 24 shows bus-off recovery from `can_mac_pcs_fce_tb`: 128 idle condition strobes from the PCS reset TEC and REC to zero and return the node to `s_error_active` (REQ-009, REQ-030).

Eleven requirements remain open. Eight are LLC requirements (REQ-001 through REQ-005, REQ-032, REQ-035, REQ-037) deferred pending implementation of `can_llc`. One P2 requirement - REQ-036 (error signaling enable) - is deferred as non-blocking. REQ-021 (error detection, P1) has partial simulation coverage: bit-error detection is exercised via recessive injection in `test_bus_off`, but stuff, form, CRC, and ACK error detection are covered by code inspection only, as each requires a frame-aware stimulus source to inject the error at the correct field boundary (Section 10.2). REQ-034 sub-claim 1 (bus re-integration before TX) remains open pending a full-stack testbench.

8.1 Testbench Results Summary

Table 8: Testbench execution status and requirements coverage.

Testbench	Requirements covered	Status
`can_mac_crc_tb`	REQ-006	Pass
`can_mac_bs_tb`	REQ-016, REQ-018	Pass
`can_pcs_tb`	REQ-024, REQ-025, REQ-026, REQ-027	Pass
`can_fce_tb`	REQ-028, REQ-029, REQ-030, REQ-034 (sub-claim 2 only)	Pass

Testbench	Requirements covered	Status
`can_mac_pcs_fce_tb`	REQ-007, REQ-008, REQ-009, REQ-010, REQ-011, REQ-012, REQ-014, REQ-015, REQ-017, REQ-019, REQ-020, REQ-021 (bit error only), REQ-022, REQ-031, REQ-033	Pass

9 Synthesis

The implemented `can_mac_pcs_fce` stack was synthesized using Quartus Prime Standard Edition 21.1.1 targeting the Cyclone 10 LP device (10CL016YU256I7G) used in Everllence's IO-extender board. The synthesis used a standalone project with all record-typed ports flattened to individual `std_logic` and `std_logic_vector` signals and all I/O marked as `VIRTUAL_PIN`, isolating logic resource consumption from I/O buffer overhead. The clock constraint was set to 6 ns (166 MHz), deliberately overconstraining relative to any realistic CAN FD system clock to expose worst-case timing paths.

9.1 Resource Utilization

The synthesized design uses 4,608 logic elements (30% of the 15,408 available on the target device) and 869 dedicated registers. No memory blocks, embedded multipliers, or PLLs are used. Table 9 shows the per-module resource breakdown.

Table 9: Resource utilization on Cyclone 10 LP (10CL016YU256I7G): CAN FD module breakdown and CAN Classic baseline.

Module	LEs	Registers	Function
`can_mac_fsm`	4,109	684	Protocol FSM and RX frame buffer
`can_pcs`	190	49	Bit timing, TDC, dual bit rate
`can_fce`	117	31	Fault confinement (TEC/REC)
`can_mac_crc`	84	53	Three parallel CRC engines
`can_mac_ser`	84	40	TX serializer
`can_mac_bs`	31	12	Bit stuffer (dynamic and fixed mode)
CAN FD total	**4,608**	**869**	
CAN Classic (`can_bus_controller`)	**1,146**	**334**	Existing controller, see Section 1.2

`can_mac_fsm` dominates at 89% of total logic elements. The primary driver is the RX frame buffer: the FSM accumulates received frames into a 70-byte (560-bit) internal byte array, and each buffer register drives combinatorial decode and mux logic. The five remaining modules together consume 506 LEs, confirming that the PCS, FCE, CRC, serializer, and bit stuffer layers add modest overhead relative to the frame buffer cost.

9.2 Timing Results

Table 10 shows the setup and hold slack across all timing corners at the overconstraining 166 MHz clock.

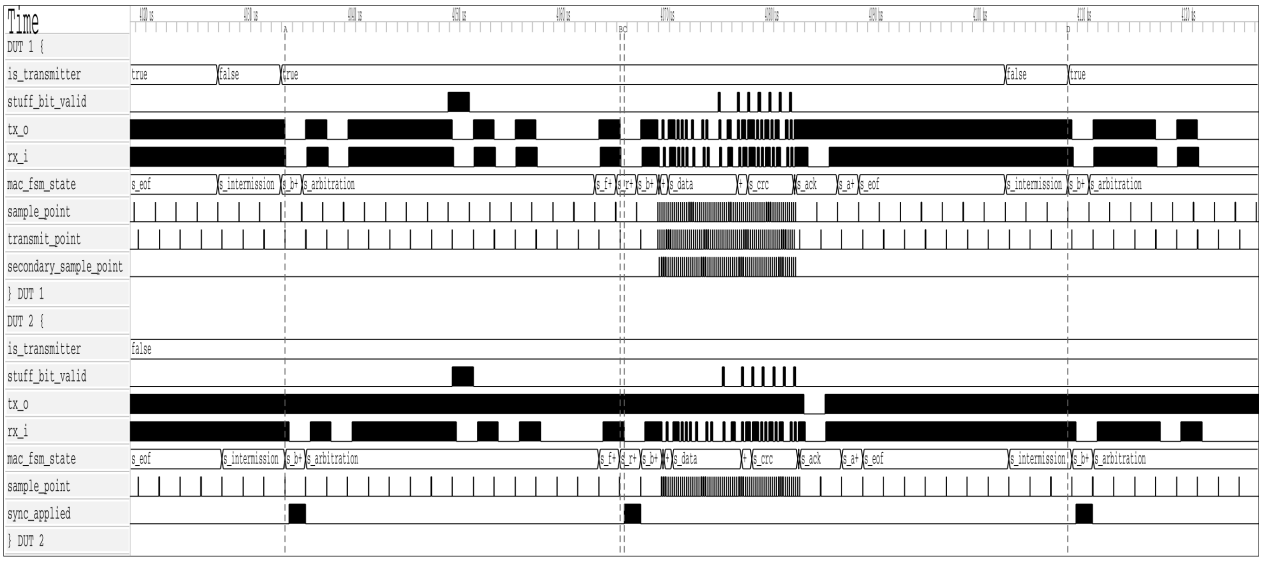

Figure 19: Two-node simulation of a complete FD frame in `can_mac_pcs_fce_tb`, showing the full field sequence from `s_arbitration` through `s_eof` on both transmitter and receiver. Secondary sample point pulses confirm TDC is active during the data phase. Resynchronization events are visible on DUT 2 via `sync_applied`.

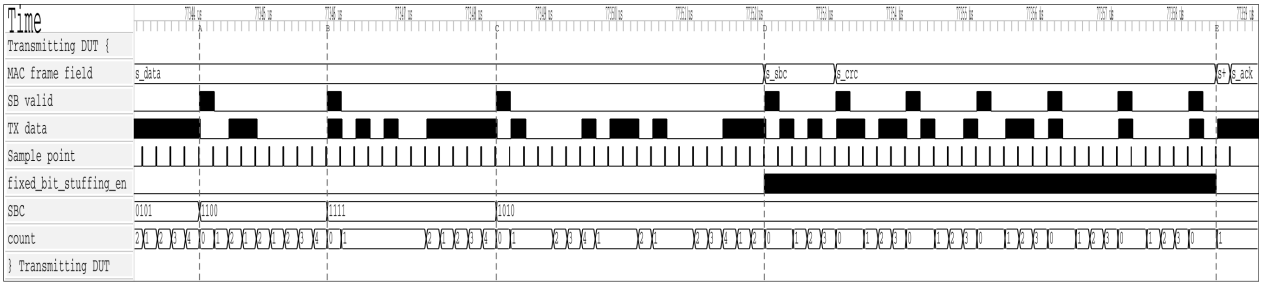

Figure 20: Dynamic and fixed bit stuffing in `can_mac_pcs_fce_tb`. Dynamic stuff bits are inserted at A, B, and C in the `s_data` region. At D, `fixed_bit_stuffing_en` asserts and the stuffer switches to fixed mode for `s_sbc` and `s_crc`. Seven fixed stuff bits are inserted between D and E.

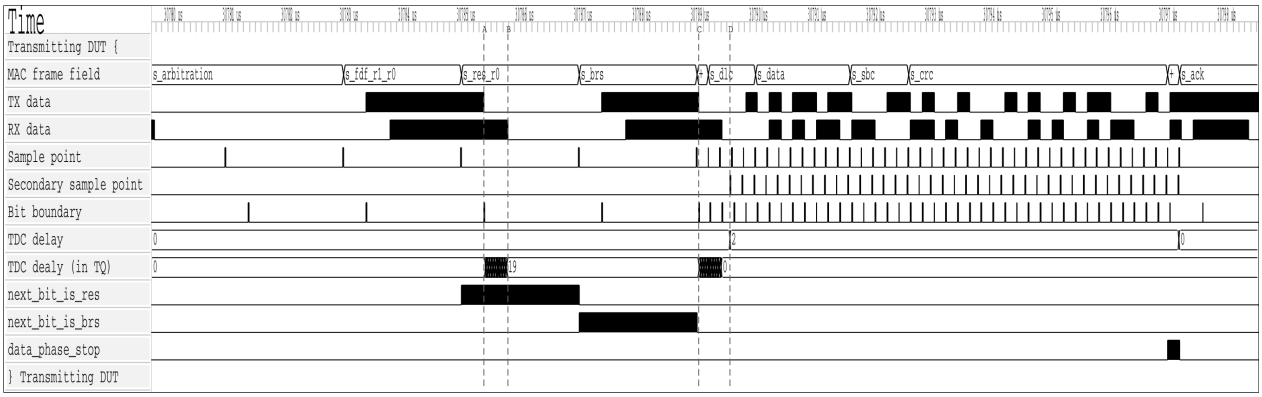

Figure 21: Dual bit rate switching and TDC measurement in `can_mac_pcs_fce_tb`. At A, the PCS begins counting the transceiver loopback delay in TQ increments. At B, the transmitted bit arrives on RX and the count stops at 19 TQ. At C, the first data-phase bit (ESI) is transmitted and the measured delay is counted down. When the countdown terminates at D, the SSP strobe activates and the TDC delay of 2 is signaled to the MAC. `next_bit_is_res` and `next_bit_is_brs` control the measurement window. `data_phase_stop` signals the end of the data phase.

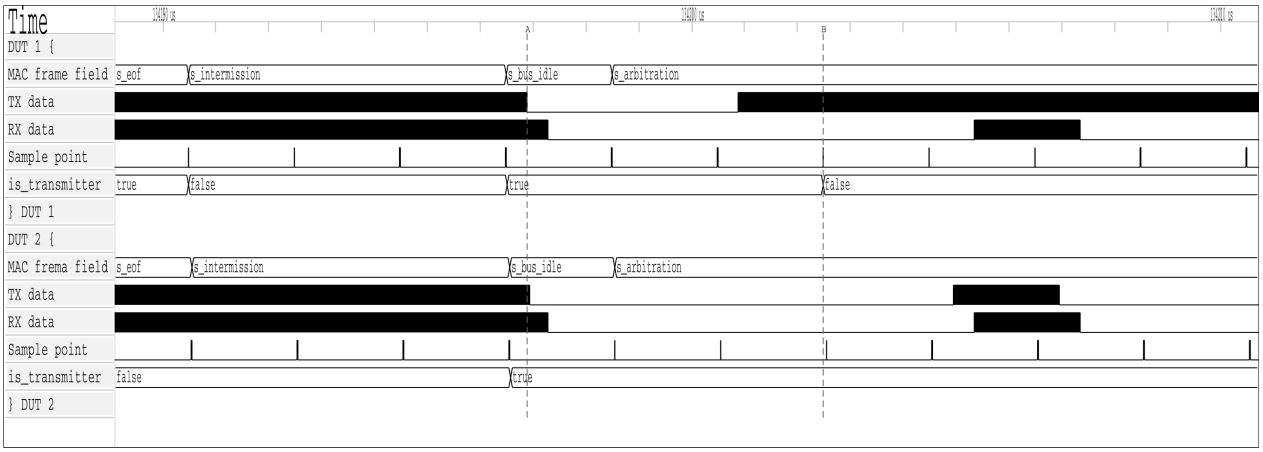

Figure 22: Arbitration loss in `can_mac_pcs_fce_tb`. Both nodes enter `s_arbitration` as transmitters at A. DUT 1 loses arbitration at B after transmitting recessive and sampling dominant, and continues as receiver with `is_transmitter` cleared.

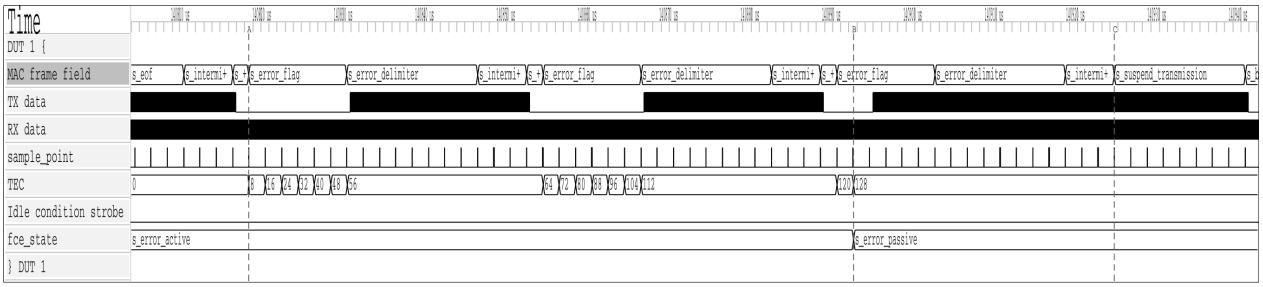

Figure 23: Error frame escalation in `can_mac_pcs_fce_tb`. A bit error on the SOF bit triggers the first error flag at A, incrementing TEC to 8. Each dominant bit of the error flag is also sampled as a bit error (bus held recessive), rapidly escalating TEC. At B, TEC reaches 128 and `fce_state` transitions to `s_error_passive`. At C, the node enters `s_suspend_transmission` after `s_intermission`.

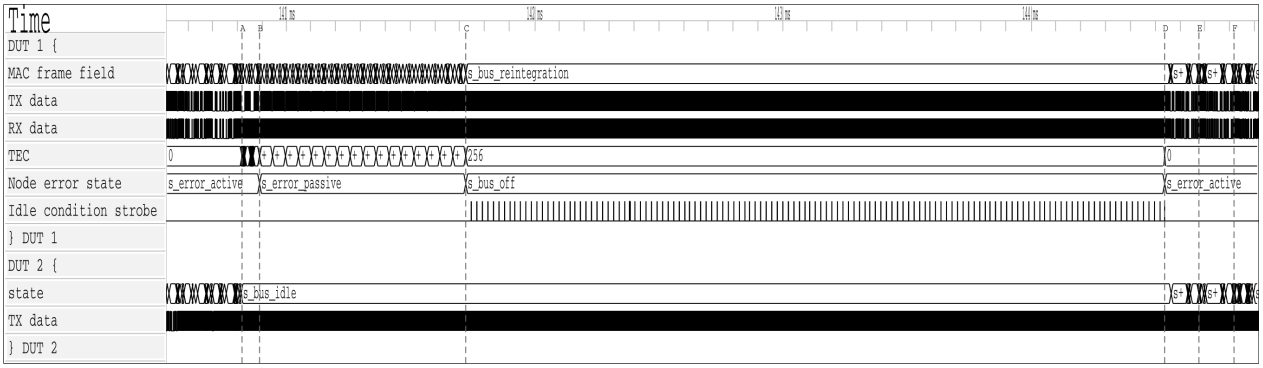

Figure 24: Bus-off recovery in `can_mac_pcs_fce_tb`. DUT 1 transmits the first active error flag at A. At B, TEC reaches 128 and the node transitions to error passive. At C, TEC reaches 256 and the node enters bus off. The FCE counts 128 idle condition strobes from the PCS and restores `s_error_active` at D. At E and F, DUT 2 acknowledges the first two frames transmitted after recovery.

Table 10: Timing results for `can_mac_pcs_fce` at 6 ns (166 MHz) on Cyclone 10 LP.

Corner	Worst setup slack	fmax estimate	Hold slack
Slow 1150 mV 100°C	−1.858 ns	~127 MHz	+0.236 ns
Slow 1150 mV −40°C	−1.550 ns	~152 MHz	+0.021 ns
Fast 1150 mV 100°C	+0.230 ns	>166 MHz	+0.163 ns
Fast 1150 mV −40°C	+0.580 ns	>166 MHz	+0.145 ns

The worst-case fmax is approximately 127 MHz on the slow 100°C corner. The setup failure at 166 MHz is a consequence of the deliberate overconstrain and does not indicate a functional problem at any realistic system clock frequency. At a 5 Mbit/s data-phase bit rate with the ISO-minimum 8 TQ per bit [8], a system clock of 40 MHz suffices at prescaler 1. The 127 MHz worst-case fmax exceeds this by more than 3×, meeting timing with comfortable margin on all corners. Hold slack is positive across all corners. At 30% device utilization on the smallest Cyclone 10 LP variant, the stack fits on the next device step up (10CL025, approximately 19%) or any larger variant in the family.

10 Discussion

The three design-facing verification plan dimensions - `layer`, `side`, and `format_applicability` - shaped the implementation in ways that were not uniformly constructive. `layer` and `format_applicability` motivated sound choices directly: the layered decomposition mapped each requirement to a testable module, and per-field FSM granularity with front-loaded config bytes followed naturally from format applicability analysis. The `side` dimension was a red herring: it made a split TX/RX architecture look well-motivated, but frame structure is the same regardless of which node is driving. Verification plan dimensions are inputs to testbench architecture - which stimulus configurations to exercise - not to RTL decomposition.

The unified FSM paid off most concretely at the arbitration loss boundary: `is_transmitter` clears in-place in `s_arbitration` and the shared CRC accumulator and bit stuffer carry over without any handoff. The split-path design required explicit state synchronization at exactly that point, and failed there first (Section 6.2). A fix to `can_mac_bs` or `can_mac_crc` propagated to both TX and RX paths automatically. Frame buffer continuity is a further payoff: the `llc_frame` byte array is already being populated from `pcs_i.rx_data` during the arbitration region, so when `is_transmitter` clears on arbitration loss, reception continues into the same buffer with no data to move. A split design would need an explicit transfer mechanism to move partially-accumulated frame data from the TX entity to the RX entity mid-frame.

The layered architecture made white-box requirements tractable. `can_fce_tb` exercised all counter-update rules in isolation. `can_mac_bs_tb` exhaustively covered stuffing mode transitions by driving the stuffer interface directly. `can_mac_pcs_fce_tb` then covered the multi-module scenarios - arbitration loss, error escalation, TDC - that cannot be observed at a single module boundary.

The AI-assisted extraction pipeline introduced a subtle bias that had direct architectural consequences. Classifying each requirement along the `side` dimension produced a requirements table organized along the TX/RX axis - a faithful representation of the ISO standard, which does frame many obligations in transmitter and receiver terms. But the artifact's structure became an implicit architectural suggestion: a table split along TX/RX lines made a split TX/RX RTL implementation look like the natural realization of the requirements model. The AI did not recommend a split architecture - it simply organized the requirements in a way that made the split appear structurally motivated. The split-path attempt was not the result of

bad engineering judgment. It followed a coherent but misleading signal from the requirements artifact. This points to a broader risk in AI-assisted engineering workflows: the structure of an extracted artifact encodes implicit suggestions about downstream decisions, and those suggestions are not labeled as such. Validating the artifact's structure - not just its content - against protocol reality is a step the AI-assisted process does not perform automatically.

The 27/38 requirement closure rate warrants careful interpretation. The 11 open requirements fall into three categories with different implications for production confidence. Eight are LLC requirements deferred pending `can_llc` implementation - a known scope boundary, not a gap in the implemented protocol engine. One P2 requirement (REQ-036 error signaling enable) is non-blocking for the core use case: a node that lacks a configurable error-signaling disable is still a fully correct CAN FD participant.

The synthesis results provide a third lens on the design. The CAN FD stack uses 4,608 logic elements on the Cyclone 10 LP target - 4.0× the 1,146 elements of the existing CAN Classic controller (Section 1.2). This growth is dominated by frame buffer scaling: the RX frame buffer grew from 120 bits (15 bytes, CAN Classic) to 560 bits (70 bytes, CAN FD), a 4.7× increase, driving the combinatorial decode and mux logic that represents the bulk of `can_mac_fsm`'s 4,109 LEs. Excluding the estimated buffer contribution, the protocol logic itself grew approximately 2.9× to deliver dual bit rate switching, TDC, three CRC polynomial variants, fixed bit stuffing with SBC, and full ISO 11898-1 fault confinement - a substantially larger feature set. Normalised for payload capacity, the FD design consumes 72 LEs per payload byte against 143 for CAN Classic, demonstrating better-than-linear scaling with respect to the 8× payload increase. The worst-case fmax of approximately 127 MHz on the slow corner exceeds the 40 MHz minimum required for a 5 Mbit/s data-phase clock at the ISO-minimum 8 TQ per bit by more than 3× (Section 9.2). The register-based frame buffer is the straightforward RTL choice but not the efficient one. At 15 bytes (120 bits) in the CAN Classic controller the address decode and read mux overhead is modest - an acceptable side-effect of the implementation approach. At 70 bytes (560 bits) both overhead terms scale proportionally, making them the dominant cost rather than a side-effect: the write-address decoder (the LUT logic that routes a byte index to the correct flip-flop's load enable) and the read mux (the LUT tree that selects the correct byte from 70 parallel register outputs) both grow with buffer depth, and at 70 entries each is large enough to dominate the module's LE count. Mapping the 70-byte buffer to a single block RAM instance would eliminate both the storage flip-flops and the combinatorial decode logic they feed, reducing `can_mac_fsm`'s footprint substantially and leaving only protocol and control logic in the LUT fabric (Section 10.2).

10.1 Objectives Assessment

The four objectives stated in Section 1.5 are assessed against the verification results.

CAN/CAN FD protocol controller in VHDL compliant with ISO 11898-1, supporting CB, CE, FB, and FE frames. The unified `can_mac_fsm` handles all four in-scope frame formats in both transmission and reception, including dual bit rate switching with Transmitter Delay Compensation in the FD data phase (REQ-024, REQ-025). 27 of 38 requirements are closed. The remaining 11 are LLC-layer requirements deferred pending `can_llc` implementation, one known P2 gap, or requirements with partial simulation coverage, all documented in Section 10.2.

ISO 11898-1 sub-layer structure enabling independent module verification. The layered decomposition was implemented as designed. Each module has a dedicated testbench and a disjoint requirement set. The five requirements labelled `system` correctly identified the scenarios requiring multi-module stimulus, confirming that the layer boundaries were drawn at the right points.

Structured requirements with traceability from ISO 11898-1 to testbench results. 38 requirements were derived from ISO 11898-1 normative clauses, each linked to its source section, verification method, testbench file, and assertion label. 27 are closed against passing testbenches or code inspection, establishing a direct traceable path from standard clause to verification artifact. The full plan is reproduced in Section C.

RTL design integrated via Avalon-ST interfaces into Everllence's existing FPGA infrastructure. The RTL source is written in portable VHDL-93 with no vendor primitives. Synthesis on a Cyclone 10 LP target confirmed timing closure at realistic system clock frequencies with 30% device utilization (Section 9). The Avalon-ST host interface is the responsibility of `can_llc`, which is not yet implemented. Its interface contracts are fully specified in the verification plan.

10.2 Future Work

1. **`can_llc` implementation.** The LLC sub-layer is the one unimplemented module. Its interface contracts are fully specified in REQ-001 through REQ-005, REQ-032, and REQ-037. Implementing it closes the Avalon-ST host interface and completes the full CAN node.

2. **Hardware integration and bring-up.** The implemented RTL has been verified in simulation only. Integration into Everllence's IO-extender FPGA design and bring-up on a physical CAN FD bus - including interoperability testing against a known-good CAN FD node - would validate timing closure, transceiver compatibility, and bit timing calibration under real bus conditions.

3. **CAN XL support.** CAN XL is explicitly out of scope for this project. The layered architecture and unified FSM are well-suited for extension: CAN XL adds a third bit rate phase and an XL-specific frame format, both of which map naturally onto additional PCS rate parameters and new `can_mac_fsm` states.

4. **Frame buffer block RAM migration.** The 70-byte RX frame buffer is implemented as a 560-bit flip-flop array, which is the primary driver of `can_mac_fsm`'s 4,109 LEs. A single M9K block RAM instance on the Cyclone 10 LP provides 9 Kbit of dedicated storage with negligible LUT overhead and is sufficient to hold the entire frame. The frame accumulation logic writes sequentially by byte index during reception, and `p_stream_to_LLC` reads sequentially during the post-EOF transfer - both access patterns map directly to single-port BRAM without any structural change to the surrounding FSM logic. This migration would reduce the LUT footprint to approximately the protocol-logic-only estimate, making the resource cost proportional to CAN FD's actual protocol complexity rather than its frame size.

5. **Error-type-specific simulation coverage (REQ-021).** The current testbench verifies bit-error detection through recessive injection at frame start. Sub-claims 2-5 (stuff, form, CRC, and ACK error detection) are covered by code inspection only. All four outstanding sub-claims require a frame-aware stimulus source. Stuff error must target a stuff bit, form error must target a fixed-format field, ACK error must target the ACK slot, and CRC error must target a non-fixed-form bit before the CRC field - an injection landing on a fixed-form bit would trigger a form error instead. Closing these sub-claims requires either the Everllence-internal CAN FD reference model with targeted error injection capability, or white-box FSM state signals exposed from `can_mac_fsm` to time the injection correctly.

11 Conclusion

This thesis presented the design, implementation, and verification of a CAN/CAN FD protocol controller in VHDL-93, structured around the ISO 11898-1 layered reference model. The implemented design covers the MAC, PCS, and FCE sub-layers as independently testable modules, supports all four in-scope frame formats (CB, CE, FB, FE), implements dual bit rate switching with Transmitter Delay Compensation, and integrates into Everllence's existing FPGA infrastructure via Avalon-ST interfaces. Of the 38 requirements derived from ISO 11898-1, 27 are closed against passing testbenches or code inspection. Of the remaining 11, eight fall outside the current scope pending `can_llc` integration, one is a lower-priority optional feature, and two represent identified future work: REQ-021 (error-injection under passive error conditions) and REQ-034 sub-claim 1 (lone-node bus re-integration). The design was synthesized on a Cyclone 10 LP FPGA target using 4,608 logic elements (30% of device) with a worst-case fmax of approximately 127 MHz, confirming timing closure at realistic system clock frequencies.

The project yielded three transferable lessons. First, the structure of a requirements model can inadvertently bias RTL architecture: the TX/RX side dimension of the verification plan made a split-path implementation appear well-motivated, but the frame structure of the CAN protocol is the same regardless of which node is driving, and cutting the natural code unit at an artificial seam added coordination complexity without reducing protocol complexity. Verification plan dimensions are inputs to testbench architecture, not to RTL decomposition. Second, the ISO 11898-1 layered architecture is not merely a documentary convenience - it is a practical partitioning of protocol complexity that, when followed in the implementation, enables each sub-layer to be implemented, verified, and debugged independently. The modular design produced here is maintainable over the long product lifecycles that motivate Everllence's decision to develop the protocol controller in-house. Third, targeted, schema-validated field updates to a structured artifact are safe for an LLM to perform incrementally. Full-file rewrites are not (Section 3.3). The narrow MCP write interface made AI-assisted maintenance of the verification plan safe and practical across all three project phases. The result is an IP core that Everllence owns outright - maintainable, verifiable, and extensible over the multi-decade service commitments that motivated the redesign.

12 References

[1] Robert Bosch GmbH, "CAN specification version 2.0," Robert Bosch GmbH, 1991.

[2] F. Hartwich, "CAN with flexible data-rate," in *Proceedings of the 13th international CAN conference (iCC 2012)*, 2012.

[3] M. Jeřábek and P. Píša, "CTU CAN FD — open-source CAN FD IP core." https://github.com/Logic-Design-Services/CTU-CAN-FD, 2019.

[4] International Organization for Standardization, *Road vehicles — controller area network (CAN) conformance test plan — Part 1: Data link layer and physical signalling*. 2016.

[5] Robert Bosch GmbH, "M_CAN — CAN FD protocol controller IP module." https://www.bosch-semiconductors.com/products/ip-modules/can-ip-modules/m-can/, 2024.

[6] AMD (Xilinx), "CAN FD controller — Vivado design suite product guide (PG223)." https://docs.amd.com/r/en-US/pg223-canfd, 2024.

[7] CAST, Inc., "CAN FD protocol controller." https://www.cast-inc.com/interfaces/automotive-bus-controllers/can-ctrl, 2024.

[8] International Organization for Standardization, *Road vehicles — controller area network (CAN) — Part 1: Data link layer and physical coding sublayer*. 2024.

[9] J. Charzinski, "Performance of the error detection mechanisms in CAN," in *Proceedings of the 1st international CAN conference (iCC 1994)*, Mainz, Germany, 1994.

[10] Texas Instruments, *TCAN1042-Q1 automotive CAN FD transceiver*. Texas Instruments, 2024. Available: https://www.ti.com/lit/ds/symlink/tcan1042-q1.pdf

[11] A. Mutter, "Robustness of a CAN FD bus system — about oscillator tolerance and edge deviations," in *Proceedings of the 14th international CAN conference (iCC 2013)*, Paris, France, 2013.

[12] A. Mutter and F. Hartwich, "Advantages of CAN FD error detection mechanisms compared to classical CAN," in *Proceedings of the 15th international CAN conference (iCC 2015)*, 2015.

[13] OSVVM Contributors, "OSVVM — open source VHDL verification methodology." https://osvvm.org, 2024.

[14] Intel Corporation, "Avalon interface specifications," Intel Corporation, 683091, 2021. Available: https://docs.altera.com/r/docs/683091/current

[15] J. Bergeron, "Verification planning," in *Writing testbenches: Functional verification of HDL models*, 2nd ed., Kluwer Academic Publishers, 2003.

A Accompanying Digital Materials

The zip file accompanying this document contains the complete source tree developed during this project. The tables below identify the key files by category.

RTL source files

File	Description
`src/can_types_p/hdl_src/can_types_p.vhd`	Shared types package
`src/can_mac/hdl_src/can_mac_fsm.vhd`	Unified MAC FSM
`src/can_mac/hdl_src/can_mac.vhd`	MAC wrapper
`src/can_mac_ser/hdl_src/can_mac_ser.vhd`	TX serializer
`src/can_mac_bs/hdl_src/can_mac_bs.vhd`	Bit stuffer/destuffer
`src/can_mac_crc/hdl_src/can_mac_crc.vhd`	CRC engine
`src/can_pcs/hdl_src/can_pcs.vhd`	PCS (bit timing, sync, TDC)
`src/can_fce/hdl_src/can_fce.vhd`	Fault Confinement Entity
`src/can_mac_pcs_fce/hdl_src/can_mac_pcs_fce.vhd`	Synthesized top-level wrapper

Testbench files

File	Description
`src/can_mac_ser/hdl_tb/can_mac_ser_tb.vhd`	Serializer
`src/can_mac_bs/hdl_tb/can_mac_bs_tb.vhd`	Bit stuffer
`src/can_mac_crc/hdl_tb/can_mac_crc_tb.vhd`	CRC engine
`src/can_pcs/hdl_tb/can_pcs_tb.vhd`	PCS
`src/can_fce/hdl_tb/can_fce_tb.vhd`	FCE
`src/can_mac_pcs_fce/hdl_tb/can_mac_pcs_fce_tb.vhd`	MAC + PCS + FCE integration

Verification plan and tooling

File	Description
`verification_plan/verification_plan.toml`	38 requirements with traceability metadata
`mcp_tools/verification_plan_manager.py`	MCP server for verification plan maintenance

B Complete CAN Node Signal Interface

Complete signal-level connectivity of the implemented CAN node. The MAC sub-layer is expanded to show its four constituent entities (`can_mac_fsm`, `can_mac_bs`, `can_mac_ser`, `can_mac_crc`). PCS and

FCE appear as external module boundaries. The LLC interface block represents the Avalon-ST boundary to the LLC sub-layer, which is not implemented in this project.

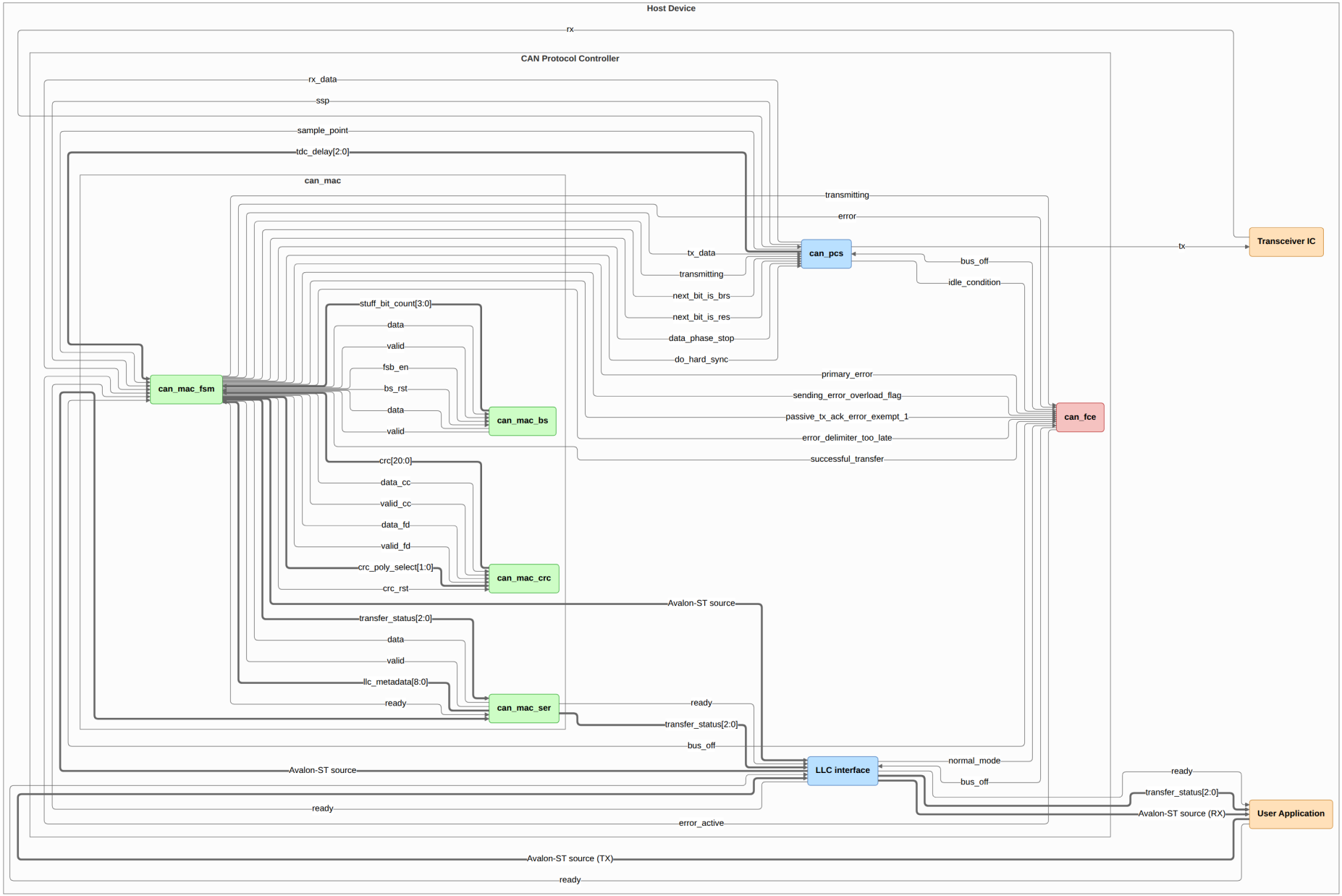

Figure 25: Complete CAN node signal-level connectivity. The MAC sub-layer is expanded to show its four internal entities. The LLC interface block is the Avalon-ST boundary to the pending LLC implementation.

C Verification Plan

Both tables are regenerated automatically from `verification_plan/verification_plan.toml` on each PDF build. The ID field is the join key between them. See Section 5.7 for the meaning of each field. The first table lists each requirement with its ISO source clause, priority, and paraphrase. The second table lists the verification metadata: layer, side, format applicability, observability, method, status, traceability label, file, and coverage criteria.

Table 11: ISO 11898-1 extracted requirements.

ID	ISO ref	Priority	Requirement	Notes
REQ-001	6.4.5.2	P1	**LLC notification** LLC shall notify the user on every successful DF/RF TX or RX.	Timestamping (§6.5.7) is optional per ISO and is out of scope.
REQ-002	6.4.5.5.2, 6.4.5.5.3	P2	**LLC transmission request and abort timing** 1) A TX request shall be processed no later than the second SOF after the request when no EFs are present. 2) Frames not yet passed to the MAC are aborted immediately. 3) An abort request shall be processed before the second SOF (no EFs present) or third SOF (with EFs present).	
REQ-003	6.5.2, 6.5.3	P1	**Retransmission** 1) Frames losing arbitration shall be retransmitted unless aborted by the LLC user. 2) Non-XL nodes shall support unlimited retransmissions.	
REQ-004	6.5.4	P3	**Frame acceptance filtering** 1) Receivers should use acceptance filtering to decide frame relevance. 2) Filtering should be a single self-contained operation with no state carry-over from previous frames.	
REQ-005	6.5.6	P1	**LLC transfer status reporting** LLC shall report five transfer status values: Ongoing, Transmitted, Lost_Arbitration, Disturbed, and Aborted, each reflecting the corresponding MAC sub-layer event.	
REQ-006	6.6.4.4	P1	**CRC rules** 1) CRC_15 for CC frames. CRC_17 for FD frames with payload ≤16 B. CRC_21 for FD frames with payload >16 B. 2) Init vector: all-zero for CRC_15. MSB-one for CRC_17 and CRC_21.	

ID	ISO ref	Priority	Requirement	Notes
REQ-007	6.6.5.2, 6.6.5.3, 6.6.6.2, 6.6.6.3	P1	**Error and overload flags** 1) Active error and overload flags: 6 consecutive dominant bits each. 2) Passive error flag: 6 consecutive recessive bits (may be overwritten by dominant bits from other nodes). 3) Error and overload delimiters: 8 recessive bits each. 4) After the flag, all nodes monitor the bus and simultaneously begin the remaining 7-bit delimiter on detecting the first recessive bit.	
REQ-008	6.6.7.1, 6.6.7.2, 6.6.7.3, 6.6.7.4, Figure 6, Figure 7	P1	**Inter-frame space** 1) Intermission shall be exactly 3 recessive bits. 2) No DF or RF may be started during intermission. 3) EFs and OFs shall not be preceded by inter-frame space. 4) Bus idle is recognised at the third recessive intermission bit (error-active nodes/receivers), the eighth recessive suspend bit (error-passive transmitters), or on leaving bus-integration state. 5) An error-passive transmitter shall suspend for 8 bit times after intermission. If another node transmits during the suspend window, the node shall become a receiver.	
REQ-009	6.6.7.5	P1	**Bus re-integration** 1) Nodes shall start bus re-integration on protocol startup or bus-off recovery. 2) Re-integration completes after 11 consecutive recessive bits at the sample point. 3) For FD-capable nodes, any synchronisation-causing edge shall reset the 11-bit recessive-bit counter.	
REQ-010	6.6.7.3, 6.6.8, Figure 10, Figure 11	P1	**Frame initiation** 1) A node shall send SOF only when the bus is idle. 2) A dominant third intermission bit causes a node with a pending frame (if error-active or previous receiver) to skip SOF and begin arbitration immediately. 3) A dominant bit detected during bus idle shall be interpreted as SOF.	

ID	ISO ref	Priority	Requirement	Notes
REQ-011	6.6.10.1, 6.6.10.4, Figure 10, Figure 11	P2	**Remote frame** RF DLC indicates the requested data length of the corresponding DF.	
REQ-012	6.6.10.2, 6.6.11.2, Figure 18, Figure 19	P1	**SRR and RRS reserved bits** 1) Receivers shall accept recessive and dominant SRR bits without form error. 2) Receivers shall accept recessive and dominant RRS bits without form error.	
REQ-013	6.6.10.5, 6.6.11.5	P1	**CRC bit stream coverage** 1) CC: SOF, arbitration, control, and data fields (dynamic stuff bits excluded). 2) FD: SOF, arbitration, control, data, dynamic stuff bits, and stuff count field (fixed stuff bits excluded).	
REQ-014	6.6.10.6, 6.6.11.6, 6.6.14, Figure 17	P1	**Frame acknowledgement** 1) Receivers shall ACK consistent frames and flag inconsistent frames with an EF. 2) Transmitters shall flag unacknowledged frames with an EF. 3) FD frames tolerate up to a 2-bit dominant ACK overlap to compensate for receiver phase differences. 4) ACK shall not depend on the frame identifier.	
REQ-015	6.6.11.3, Figure 20, Figure 21	P2	**ESI bit transmission** 1) Recessive when the LLC ESI flag is set. 2) Dominant when LLC ESI flag is clear and the node is error-active. 3) Recessive when LLC ESI flag is clear and the node is error-passive.	
REQ-016	6.6.11.5	P1	**Stuff bit count (SBC) field** 1) SBC: 3-bit Gray-coded count of dynamic stuff bits (0-7) plus parity bit, placed before the CRC field. 2) TX shall transmit its dynamic stuff-bit count (up to the first FSB) as the SBC field. 3) RX shall verify the received SBC against its own count.	

ID	ISO ref	Priority	Requirement	Notes
REQ-017	6.6.10.5, 6.6.11.5	P1	**CRC delimiter** 1) CC frames: exactly one recessive CRC delimiter bit. 2) FD frames: transmitter sends one recessive bit. A second recessive bit is tolerated before the ACK slot.	
REQ-018	6.6.13.1, 6.6.13.2, 6.6.13.3.1	P1	**Bit stuffing** 1) After 5 consecutive identical bits, insert a complementary dynamic stuff bit. RX discards it. 2) CC: dynamic stuffing spans SOF to FCRC. FD: SOF to end of data field. ACK and EOF are not stuffed. 3) FD CRC field: a fixed stuff bit precedes the SBC field and follows every fourth CRC bit, each the inverse of the preceding bit. No double stuffing. 4) RX discards FSBs and flags a form error if an FSB matches the preceding bit.	
REQ-019	6.6.10.7, 6.6.11.7, 6.6.15.2, Figure 8, Figure 9	P1	**End of frame (EOF)** 1) A receiver shall validate a frame after the sixth EOF bit. 2) A transmitter requires all seven EOF bits to be error-free.	
REQ-020	6.6.10.2, 6.6.11.2, 6.6.17.4, 6.6.17.5, Figure 10, Figure 11, Figure 18, Figure 19	P1	**Bus arbitration** Recessive-sent, dominant-monitored: lose arbitration and become a receiver.	

ID	ISO ref	Priority	Requirement	Notes
REQ-021	6.6.21.2	P1	**MAC error detection** 1) Bit error: sent/monitored mismatch. Exceptions: recessive non-stuff bit during arbitration, recessive bit in ACK slot, dominant bit while sending a passive error flag. 2) Stuff error: sixth consecutive identical bit in a dynamically-stuffed field. 3) Form error: unexpected fixed-form bit value, or FSB not inverse of preceding bit (FB/FE). Exception: dominant last EOF bit or last delimiter bit is not a form error. 4) CRC error: CRC mismatch. FD also requires SBC match and correct parity. 5) ACK error: transmitter detects no dominant bit in the ACK slot.	Sub-claim 1 (bit error) is simulation-verified via recessive injection in test_bus_off. Sub-claims 2-5 require frame-aware error injection not available in the current testbench. All four are covered by code inspection only.
REQ-022	6.6.21.3.1, Figure 28, Figure 29	P1	**Error flag initiation and CRC error behaviour** 1) On any detected error, start an EF at the next bit and inform the LLC. 2) Data-phase errors: switch to nominal bit rate before starting the EF. 3) CRC error: receiver withholds ACK and sends an EF at the first EOF bit (CC) or 3 bit-times after the CRC delimiter (FD).	
REQ-023	6.6.21.3.2	P1	**Overload frame** 1) LLC-requested OF: optional, starts at the first intermission bit, at most 2 per inter-frame space. 2) Reactive OF: triggered by a dominant bit at the first two intermission bits, the last EOF bit (receivers only), or the last error/overload delimiter bit. Starts one bit after the trigger.	

ID	ISO ref	Priority	Requirement	Notes
REQ-024	7.3.2, 7.3.3, Figure 32	P1	**PCS bit timing configuration** 1) TQ is a fixed time unit derived from the node clock period. TQ = m × t_q,min where m is a programmable integer prescaler and t_q,min = one clock period. 2) A bit time consists of four segments: Sync_Seg (1 TQ), Prop_Seg (compensates for bus propagation and node delays), Phase_Seg1, and Phase_Seg2. The sample point falls at the end of Phase_Seg1. 3) IPT (Information Processing Time): the time a node requires to determine the bit value after sampling. IPT ≤ 2 TQ. 4) Separate nominal and FD data bit rate configurations, each based on a programmable integer prescaler.	Deployment constraints on the system, not RTL behaviors: prop_seg, SJW, Phase_Seg2, and oscillator tolerance must be programmed to meet the bounds in §7.3.2, §7.3.3, and §7.3.6. Minimum configuration ranges are given in Table 13 (§7.3.3).
REQ-025	7.3.4, 6.6.21.3.1, Figure 34, Figure 35	P1	**Transmitter delay compensation** 1) TDC applies in the FD data phase when BRS is recessive. Enable and prescaler (1 or 2) are programmable. 2) SSP position = measured transmitter delay + configured offset. 3) TDC active: bit errors at the SP are ignored. SSP errors are flagged at the following SP.	

ID	ISO ref	Priority	Requirement	Notes
REQ-026	7.3.5.1, 7.3.5.2, 7.3.5.3, 7.3.5.4, Figure 33	P1	**PCS synchronisation** 1) Phase error: the time interval between a recessive-to-dominant edge and the Sync_Seg boundary of the current bit time. Positive when the edge falls after Sync_Seg (late), negative when before (early). 2) SJW (Synchronization Jump Width): the maximum amount by which Phase_Seg1 or Phase_Seg2 may be adjusted per resynchronization. 3) One synchronisation per bit time, re-enabled after the next recessive SP. 4) Only recessive-to-dominant edges qualify. An edge qualifies only if the previous SP was recessive and no positive phase error exists while sending dominant. 5) Hard sync: IFS edges (except first intermission bit), bus-integration, and FDF-to-res transition. Restarts bit time with Sync_Seg completed, unconstrained by SJW. 6) All other qualifying edges cause resync, except for the transmitter during the FD data phase. Positive error: Phase_Seg1 += SJW. Negative error: Phase_Seg2 -= SJW. Error ≤ SJW: same effect as hard sync.	RTL is stricter than ISO: mac_i.transmitting suppresses all sync unconditionally, not just resync on positive phase error. Safe since the transmitter is the timing source.
REQ-027	6.6.7.5, 7.4.2.2, 7.4.2.3, 8.1.4.4	P1	**PCS bus-off isolation and signalling** 1) During bus-off, the PCS shall not drive dominant bits. 2) PCS enters bus-off on a supervisor request and exits on a release request.	
REQ-028	8.1.3.1, 8.1.3.2, Table 14, Table 15, Figure 42	P1	**FCE reset and response** 1) TEC and REC shall both be reset to zero when the FCE enters its initial state.	

ID	ISO ref	Priority	Requirement	Notes
REQ-029	8.1.4.2 rule a), rule b), rule c), rule d), rule e), rule f), rule g), rule h)	P1	**Error counter rules** 1) REC +1 on any receiver-detected error (except bit errors during error/overload flag). 2) REC +8: first dominant bit after sending an error flag, or bit error while sending an active error/overload flag. 3) REC -1 on successful RX if 1-127. Stays 0 if already 0. Set to 119-127 if above 127. 4) TEC +8 when sending an error flag. Exceptions: error-passive ACK error, pre-arbitration stuff error. 5) TEC +8 on bit error while sending an active error or overload flag. 6) TEC -1 on successful TX (floor 0). 7) After 14 consecutive dominant bits post active error/overload flag (8 post passive error flag): both +8. Each additional 8 dominant bits: both +8.	
REQ-030	8.1.4.3, 8.1.4.4, 8.1.3.2, Figure 43	P1	**Error confinement state transitions** 1) Error-passive when either error counter > 127. Error-active restored when both ≤ 127. 2) TEC > 255 causes bus-off. 3) Bus-off recovery requires 128 idle conditions. Both counters reset to zero on recovery. 4) FCE shall notify the LLC on entering bus-off state.	Sub-claim 4: bus-off recovery requires 128 separate idle detections (128 × 11 recessive bits), not 128 consecutive recessive bits.
REQ-031	6.6.11.3, Figure 20, Figure 21	P1	**BRS bit rate switching** 1) Recessive BRS: switch to FD data rate at the BRS sample point. Dominant BRS: keep nominal rate. 2) Rate switches back to nominal at the first CRC delimiter sample point.	
REQ-032	6.4.3, 6.4.4, Table 5	P1	**DLC byte-count mapping** DLC 0-8: linear mapping. FD: DLC 9⟶12, 10⟶16, 11⟶20, 12⟶24, 13⟶32, 14⟶48, 15⟶64 bytes. CC: DLC 9-15 clamped to 8 bytes.	

ID	ISO ref	Priority	Requirement	Notes
REQ-033	6.6.16	P1	**Byte/Bit transmission order** Within the data field byte 0 is sent first. Within each byte bit(7) is sent first.	
REQ-034	8.1.5	P1	**Node start-up** 1) A node shall complete bus re-integration (11 consecutive recessive bits) before transmitting. 2) A single node receiving no ACK becomes error-passive but never bus-off.	Sub-claim 2 verified via FCE Exception 1 mechanism (passive_tx_ack_error_exempt_1) in can_fce_tb.
REQ-035	6.6.18	P1	**MAC data consistency** Frames shall be fully buffered before transmission starts.	
REQ-036	6.6.21.1, 6.6.21.4	P2	**Error signalling enable** A configuration parameter shall exist to enable or disable error signalling.	
REQ-037	6.4.4	P3	**DLC padding** An LLC restricted to fewer bytes shall replace excess bytes with 0xCC padding in both TX and RX directions.	Only applicable if the implementation exposes a configurable maximum-data-byte restriction. If not implemented, waive.

ID	ISO ref	Priority	Requirement	Notes
REQ-038	5.2, Figure 2	P1	**Frame field sequence** 1) CBFF DF: SOF (D) → Arbitration (ID[28:18], RTR (D)) → Control (IDE (D), r0 (D), DLC[3:0]) → Data → CRC (CRC_15, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R). 2) CBFF RF: as CBFF DF but RTR (R) and no data field. 3) CEFF DF: SOF (D) → Arbitration (ID[28:18], SRR (R), IDE (R), ID[17:0], RTR (D)) → Control (r1 (D), r0 (D), DLC[3:0]) → Data → CRC (CRC_15, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R). 4) CEFF RF: as CEFF DF but RTR (R) and no data field. 5) FBFF DF: SOF (D) → Arbitration (ID[28:18], RRS (D)) → Control (IDE (D), FDF (R), res (D), BRS, ESI, DLC[3:0]) → Data (FD rate) → CRC (SBC[3:0], CRC_17/21, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R). 6) FEFF DF: SOF (D) → Arbitration (ID[28:18], SRR (R), IDE (R), ID[17:0], RRS (D)) → Control (FDF (R), res (D), BRS, ESI, DLC[3:0]) → Data (FD rate) → CRC (SBC[3:0], CRC_17/21, delimiter (R)) → ACK (slot, delimiter (R)) → EOF (7×R).	Notation: (D) = dominant, (R) = recessive. Unannotated bits (ID, DLC, BRS, ESI, CRC value, SBC, ACK slot) are data-dependent or conditional.

Table 12: ISO 11898-1 verification plan.

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-001	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-002	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-003	LLC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-004	LLC	receiver	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-005	LLC	transmitter	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-006	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	crc_15, crc_17, crc_21	can_mac_crc_tb	Three bins: (1) CB or CE frame (CRC_15), (2) FB or FE frame with data ≤ 16 B (DLC ≤ 10, CRC_17), (3) FB or FE frame with data > 16 B (DLC ≥ 11, CRC_21) - must each be hit.
REQ-007	system	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~900, test_bus_off	can_mac_pcs_fce_tb	
REQ-008	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~486, test_normal, test_bus_off	can_mac_pcs_fce_tb	
REQ-009	system	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~433, test_bus_off	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-010	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~451, test_normal	can_mac_pcs_fce_tb	
REQ-011	MAC	both	CB, CE	black_box	simulation	complete	FTYP Coverage	can_mac_pcs_fce_tb	
REQ-012	MAC	both	CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~546, p_fsm:~548, test_normal	can_mac_pcs_fce_tb	
REQ-013	MAC	both	CB, CE, FB, FE	white_box	code_inspection	complete	p_fsm:~299, p_fsm:~216	can_mac_fsm	
REQ-014	system	both	CB, CE, FB, FE	white_box	simulation, wave-form_inspection	complete	test_normal	can_mac_pcs_fce_tb	
REQ-015	MAC	transmitter	FB, FE	white_box	simulation, wave-form_inspection	complete	test_normal	can_mac_pcs_fce_tb	
REQ-016	MAC	both	FB, FE	black_box	simulation	complete	p_sbc_checker	can_mac_bs_tb	
REQ-017	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~768, p_fsm:~796, test_normal	can_mac_pcs_fce_tb	
REQ-018	MAC	both	CB, CE, FB, FE	white_box	simulation, coverage	complete	Input Coverage, Output Coverage	can_mac_bs_tb	All bins in cov_input and cov_output must be hit.
REQ-019	MAC	both	CB, CE, FB, FE	white_box	simulation	complete	test_normal	can_mac_pcs_fce_tb	
REQ-020	system	transmitter	CB, CE, FB, FE	black_box	simulation	complete	test_lost_arb	can_mac_pcs_fce_tb	
REQ-021	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	in_progress	p_fsm:~283, p_fsm:~336, p_fsm:~343, p_fsm:~415, test_bus_off	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-022	MAC	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~283, p_fsm:~294, p_fsm:~357, test_bus_off	can_mac_pcs_fce_tb	
REQ-023	MAC	both	CB, CE, FB, FE	white_box	code_inspection	complete	p_fsm:~370	can_mac_fsm	
REQ-024	PCS	both	CB, CE, FB, FE	white_box	simulation	complete	test_normal	can_pcs_tb	
REQ-025	PCS	transmitter	FB, FE	black_box	simulation	complete	p_check_tdc_delay	can_pcs_tb	
REQ-026	PCS	both	CB, CE, FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_can_pcs:~177, p_can_pcs:~228, test_normal	can_pcs_tb	
REQ-027	PCS	both	CB, CE, FB, FE	white_box	simulation, code_inspection	complete	p_can_pcs:~280, p_can_pcs:~379, test_bus_off	can_mac_pcs_fce_tb	
REQ-028	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_reset, test_bus_off_recovery	can_fce_tb	Sub-claim 1: TEC/REC reset shall take effect within one clock cycle of the request. Sub-claim 2: verify that Nor-mal_mode_response is asserted after reset completes.

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-029	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_rule_a, test_rule_b, test_rule_c, test_rule_c_exception, test_rule_d, test_rule_e, test_rule_f_tx, test_rule_f_rx, test_rule_g, test_rule_h	can_fce_tb	Rule h: the '>127 ⟶ 119-127' clamp path must be explicitly confirmed in simulation.
REQ-030	FCE	both	CB, CE, FB, FE	black_box	simulation	complete	test_bus_off, test_bus_off_recovery	can_fce_tb	
REQ-031	MAC	both	FB, FE	white_box	code_inspection, wave-form_inspection	complete	p_fsm:~636, p_fsm:~755, test_normal	can_mac_pcs_fce_tb	
REQ-032	LLC	both	CB, CE, FB, FE	black_box	simulation, coverage	not_started			Four non-linear FD DLC values (9, 12, 14, 15 ⟶ 12, 24, 48, 64 bytes) plus DLC=8 boundary must each produce a correctly sized frame.
REQ-033	MAC	both	CB, CE, FB, FE	black_box	simulation	complete	test_normal	can_mac_pcs_fce_tb	

ID	Layer	Side	Format	Observability	Method	Status	Label	File	Coverage criteria
REQ-034	system	both	CB, CE, FB, FE	black_box	simulation	complete	test_rule_c_exception; normal_test, test_bus_off (waveform)	can_mac_pcs_fce_tb	Sub-claim 1: confirm no TX before 11 consecutive recessive bits are detected at the SP. Sub-claim 2: single-node scenario with no ACK injection; TEC must reach error-passive but not exceed 255.
REQ-035	MAC	transmitter	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-036	MAC	both	CB, CE, FB, FE	white_box	code_inspection	not_started			
REQ-037	LLC	both	CB, CE, FB, FE	black_box	simulation	not_started			
REQ-038	MAC	both	CB, CE, FB, FE	white_box	waveform_inspection	complete	test_normal	can_mac_pcs_fce_tb	